

TECHNICAL UNIVERSITY OF DENMARK



42417 Simulation in Operations Management
-
Course notes

Edited by:

Allan LARSEN

DTU MANAGEMENT

May 29th, 2026

Abstract

This document has been compiled to serve as an additional learning resource for students of the course 42417 Simulation in Operations Management.

First, the guide introduces content not directly accessible via the AnyLogic Help-guide. It is assumed that the reader has completed the Discrete-event-modelling part of the AnyLogic in Three Days" guide and has a basic understanding of statistics.

The guide will explain the basics of the logic used when implementing models in AnyLogic and continue with more advanced topics within AnyLogic. Topics included will be Stochastic and Deterministic input and how to process the output to apply an analytical view.

To help the reader understand the concepts, a further development of the Job Shop example from AnyLogic in Three Days" has been made and will, from here on, be mentioned as the Job Shop example.

The document has been authored and developed since May 2018, and it was initiated by former MSc students Andreas Salk and Frederik Toftegaard. Andreas and Frederik: A huge thanks to you for your hard work in making the first version of the 42417 course a success and for your eagerness to compile the first version of this valuable document. Next, former exchange student Riccardo Aldrighetti, now PhD graduate from the University of Padova, is thanked for his valuable inputs on especially the first version of the output analysis chapter and the work on parameter variation. Grazie mille, Riccardo! The 2021 version of the notes, were updated with a much more elaborated section on output analysis authored by former MSc student Frederik Fynbo Carlsson. This section presents methods to compute the number of replications needed to ensure confidence in the simulation experiments. Thanks a lot, Frederik, for authoring this important part of the guide! In the newest version, we have added a section on "Experimental Design" providing insights into how to design the experimental set-up of your simulation study when the model has been verified and validated. Former MSc students Sofie Amalie Lundsten and Katinka Schjeldrup Klepsch authored the first version of this last section and the appendix example on the YummiBar's production line. Thanks, a lot, Sofie + Katinka!

Finally, I would like to thank all of the Teaching Assistants and, last but not least, the students for assisting, cheering and comforting each other when things have been challenging... as they quite often tend to when working on simulation modeling.

Best regards,

Allan Larsen, Professor

Contents

1. Logic and loops using Java	5
1.1. if statements	5
1.2. for-loop	6
1.3. while-loop	7
2. Process Model Library	8
2.1. Source / Sink	8
2.2. Resource Pool	8
2.3. Seize / Release	9
2.4. Queue	9
2.5. Delay	9
2.6. Move To	10
2.7. Hold	10
2.8. Split / Combine	11
2.9. Assembler	11
2.10. Service	11
2.11. Resource Task Start / End	12
2.12. Time Measure Start / End	12
2.13. Restricted Area Start / End	12
2.14. Pickup / Dropoff	13
2.15. Rack Store / Pick	13
2.16. Select Output	14
3. Input Modeling	15
3.1. Deterministic input	15
3.1.1. Arrival table / Database	15
3.2. Stochastic input	17
3.2.1. Normal	18
3.2.2. Triangular	18
3.2.3. Poisson	19
3.2.4. Rate and Rate Schedule	19
3.3. Parameters and variables	21
3.3.1. Parameters from databases	22
4. AnyLogic tools, blocks and functions for Output Analysis	24
4.1. Statistical calculations	25
4.1.1. Output block	25
4.1.2. Data set	26
4.1.3. Excel files	27
4.1.4. Functions	29

4.2. Visualizations	30
4.2.1. Time plot	31
4.2.2. Bar Plot	32
4.3. Analyzing output	34
5. Output Analysis using replications	37
5.1. Using replications will increase the credibility of a simulated output	37
5.1.1. Random noise in the modified Job Shop tutorial	37
5.1.2. True performance of the modified Job Shop tutorial	38
5.2. Required number of replications	39
5.2.1. Half-width method by Rossetti	39
5.2.2. Iterative process by Ross	40
5.2.3. Single formula derived from both Rossetti and Ross	41
5.2.4. Determining the number of replications - Practical Approach	43
5.3. Comparison of system configurations	44
6. Experimental design	46
6.1. What is experimental design?	46
6.1.1. The operations management (OM) perspective	46
6.1.2. The purpose of experimental design	46
6.2. Creating an experimental design	47
6.2.1. Scenarios	47
6.2.2. OFAT approach, factorial designs and optimisation	48
6.3. How-to plan the experimental design	49
6.3.1. How-to guide	49
7. Further reading	51
A. Simulation project - example	53
A.1. Setting the objectives	54
A.2. Base case scenario	54
A.3. Scenario 1 - Predictive maintenance on wrapping machine	57
A.4. Scenario 2 - New moulding machine	58
A.5. Scenario 3 - remove cardboard box machines	58
A.6. Scenario 4, combination of scenario 1 and 2	59
A.7. Scenario 5, combination of scenario 2 and 3 (moulding and cardboard box)	59
A.8. Comparison of scenarios	60
B. Simplified activity diagram for the modified JobShop model used in chapter 5	61

1. Logic and loops using Java

AnyLogic is a simulation tool programmed using Java. This does not mean that the user is supposed to be a skilled programmer to use **AnyLogic**. All the advanced coding has already been done in the blocks, the user is using to create their model. However in some cases, the blocks themselves are not enough, and the user needs to specify what is needed to run the model as intended. To do this an *if-statement* might be needed when an agent is leaving or entering a block. When creating functions a *for-* or *while-loop* might be needed to calculate the result the user wants to present. The following sections will go over the basics of *if-statements*, *for-loops* and *while-loops*. If you are skilled in programming or have used Java before then this section is skippable.

1.1. if statements

An if statement is a basic way of controlling the flow in your model. Often in coding, it is desired to run a section of code depending on whether or not a condition is true. When dealing with if statements there are 2 main types: IF-THEN and IF-THEN-ELSE. IF-THEN statements run the wanted section of code if the condition is true, and IF-THEN-ELSE run the wanted section of code if the condition is true, however, if the condition is false, then it will run the code written in the else section. The syntax of the if-statement is shown below:

```
if (condition){
    //code if condition is true
}else{
    //code if condition is false
}
```

When writing statements and loops, certain operators might be needed to state the condition correctly. Below, the most common operators are explained, and the Java syntax is shown:

Operators	
<, >, >=, <=	Greater, lower, greater or equal and lower or equal.
==	Compares if 2 expressions are exactly the same, and will be true if they are (e.g. 2 == 2 will return true and "2" == 2 will return false, because "2" is a string)
!=	Compares if 2 expressions are not the same ("2" != 2 will return true)
	The or operator compares if 1 of 2 or both statements are true.
&&	The and operator compares if 2 conditions are true or not (e.g 2 == 2 && "2" != 2 will return true)
%	The Modulus operator is known as the remainder operator. Meaning it will return the remainder of 2 numbers. (e.g. 10 % 3 = 1. 10-3-3-3=1). This is not a boolean operator like the others, but very useful in some cases.

To show how an if-statement can work, an example from a "hold" block in the Job Shop example is shown:

```

if ( self.in.count() % BlueAssemblerCap == 0 ){
    self.setBlocked(true);
}

```

What this code does is that it blocks itself (**self.setBlocked(true)**) if the remainder of the amount of agents (**self.in.count()**) and the capacity of blue agents (**BlueAssemblerCap**) is 0. This is useful if a block cannot contain more than a certain amount of agents while processing. This way the processing block will not get overloaded and crash the simulation. However in this case you also need to unblock it when the agents leave the processing block, otherwise the whole system will be blocked (**holdBlockName.setBlocked(false)**).

1.2. for-loop

A for-loop will most likely be used to calculate an output that **AnyLogic** does not produce. To make a for-loop an initialization needs to be made, which typically is the index of an array the loop starts in. This number keeps getting updated every time the loop has run once. The loop will keep running until the specified condition no longer holds.

```

for(initialization; condition; update) {
    // Statements
}

```

An example of a for-loop can be written as:

```

int result = 0;
for(int i = 0; i<Data.size(); i++){
    result += Data.getY(i);
}
return result;

```

This for-loop from **AnyLogic** calculates the sum of a data set created in **AnyLogic** (**Data**). The loop will run until "i" is equal to the number of the size of the data set (**Data.size()**). "i" will start at 0 (**int i = 0**). 0 is the starting index of an array in Java, not 1. "i" will increase by 1 each time the loop has run once (**i++**). "result" is the sum of the data set, and it is calculated by using the += operator, that adds the previous value of "result" + the new value given from the data set in the i'th place (**Data.getY(i)**). This is just another way of writing $result = result + Data.getY(i)$.

i	Y	Result		
0	23	Result[0]	Result[-1]+Y[0]	0+23=23
1	24	Result[1]	Result[0]+Y[1]	23+24=47
2	43	Result[2]	Result[1]+Y[2]	47+43=90
3	52	Result[3]	Result[2]+Y[3]	90+52=142
4	11	Result[4]	Result[3]+Y[4]	142+11=153

Figure 1.1.: Visualization of the SumFunction running through a table column

From Figure 1 we see that **result** starts at 0 and then adds the first value of **Y**, therefore, **result** equals 23. Then it adds the next value of **Y** and keeps on running until **i** is the same number as the size of the data set.

1.3. while-loop

A while-loop is another way of writing a loop. If you know the exact number of times you want to run the loop a for-loop is preferred, however, a while-loop is preferred if the exact number is not known. A while loop runs until the boolean expression written in the loop is no longer true.

```
while( Boolean_expression ) {  
    // Statements  
}
```

However, in this course, a for-loop is the most useful. To give an example of how a while-loop can be used, the sum function from above is written in a while-loop:

```
int result = 0;  
int i = 0;  
while(i < Data.size()){  
    result += Data.getY(i);  
    i++;  
}  
return result;
```

2. Process Model Library

In this section, an in-depth explanation of the blocks in the process model library in [AnyLogic](#) is shown. The library is the basis of the simulation, and it is in this section of the palette you can change how the system acts. We will discuss the most used parts of the process model library and give examples of these.

2.1. Source / Sink

The **Source** and the **Sink** is a need for a simulation to run. The source sets the input and how it acts. While the sink ensures that the entities leave the system again. How to alter the input of the source will be described in section 4 Input.



Figure 2.1.: Source



Figure 2.2.: Sink

The source has one port, the out port which defines where the entity goes after being loaded into the system. The sink is the antagonist of the source and has an in port which defines the last step before the entity leaves the system. If you want more than one entity to start the system, e.g. different products, multiple sources have to be made. This is done in the job shop example with three pallets entering the system, RedPallet, GreenPallet and BluePallet.

2.2. Resource Pool



Figure 2.3.: Resource Pool

The **Resource Pool** defines a set of agents that can be used to perform a given task in the model. The resource pool's agents can be static, moving or portable. A static resource is like the CNC machines and a moving resource is like the forklifts in the Job Shop example. When creating a resource pool defining the capacity is an important task. This can be done by selecting areas in your model where the agents are allocated (e.g. CNC machines) or the capacity can be explicitly defined (e.g. number of forklifts). A schedule can also be used to define the capacity of your resource pool (e.g. day and night shifts in a hospital).

2.3. Seize / Release

The **Seize** block locks up a given resource from the resource pools. The seize is useful when simulating different actions or processes in **AnyLogic**, e.g. when moving an entity and a certain resource is needed to perform this action. The **Release** block then specifies when the seized agent is done with the task and returns to the agent's resource pool.

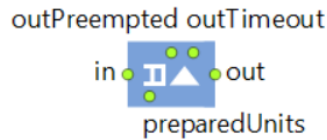


Figure 2.4.: Seize



Figure 2.5.: Release

In the Job Shop example, this is done several times. The earliest example is when the three pallets have to be moved from the pallet rack to the CNC machines. Here a CNC machine is seized, then the pallet is moved to the CNC machine and the pallet is processed. When the process is done the CNC machine is released and the pallet can move on.

2.4. Queue



Figure 2.6.: Queue

The **Queue** block is used as a buffer for the agents waiting to go further into your model. A queue can have different priorities regarding which agent leaves the queue first. There is FIFO (first in first out), LIFO (last in first out) and priority based. Your agents can have certain parameters that make them more important than others, and for this reason, you could use the priority-based queue.

2.5. Delay



Figure 2.7.: Delay

The **Delay** block is used to delay an agent a certain amount of time. This can be both stochastic or written explicitly. In a model, it can be used as waiting time or a way to simulate the amount of time it takes to process an agent (e.g. processing pallets in a CNC machine). A delay block can

also depend on a "stopDelayForAll()" being called in another block (e.g. if the queue size in a "queue" block is 0 a stopDelayForAll() can be called).

2.6. Move To



Figure 2.8.: Move To

The **Move To** block simply moves the agent and all attached resources to a new given location. The location can be a specified node in the model, or the agent can be moved to a seized resource unit, like the CNC machines in the Job Shop example.

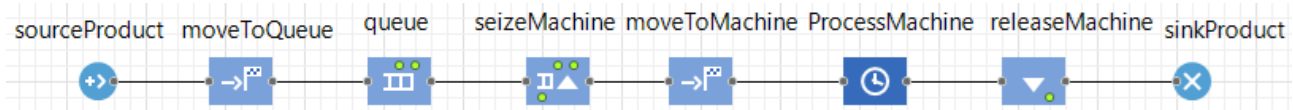


Figure 2.9.: Example of blocks

Above you see an example of a small production. The system has some entities coming into the system (**sourceProduct**), and are then moved to a queue (**moveToQueue** , e.g. a storage space), from there (**queue**) on it waits until the machine is free / can be seized (**seizeMachine**). When the machine is free the entity goes into the machine, and after the delay (**ProcessMachine**) the system releases the machine so other entities can go in.

2.7. Hold

The **Hold** block can be used to stop the agent flow in your model if needed. The hold block cannot contain any agents and should be seen as an extension of other blocks (e.g. keep the agents in a queue). To change the state so that the agents are blocked setBlocked(true) must be called, and to unblock it setBlocked(false) must be called. This is typically done when an agent leaves or enters a specified block.



Figure 2.10.: Hold

To show how this can be used look in the Job Shop example. Here the hold is used to stop the pallets from entering the Assembler if the capacity of the pallets in the Assembler has been reached. The simulation will stop if the amount of a certain agent is above its capacity in the Assembler, and for this reason, the "hold" block is needed.

2.8. Split / Combine

The **Split** and **Combine** blocks either split or combine entities into multiples of the given entity or a new entity.

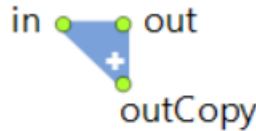


Figure 2.11.: Split

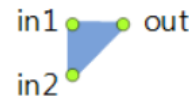


Figure 2.12.: Combine

In the Job Shop example, we see that the clamps splits into a clamp and some waste material. A different example could be done by combining two materials into a new one.

2.9. Assembler

The **Assembler** block is an expanded combine block, a seized block and a delay block. In the assembler, you can combine more than two entities.



Figure 2.13.: Assembler

In the Job Shop example it can be seen that the model combines three different pallets into a new product, the clamp. However, we also specify that this process takes a specific amount of time and uses a worker as its resource to perform the action.

2.10. Service

The **Service** block has 3 functions in one block. It has the same function as having a "Seize", "Delay" and "Release" block in that order. For this reason, it should be used when nothing happens between these 3 blocks. A waiting place can also be specified in the Service block.

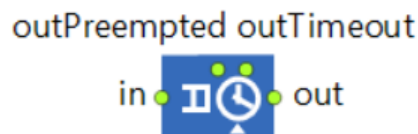


Figure 2.14.: Service

In the Job Shop example, it is used to paint the clamps after the clamps have been assembled in the "Assembler". Here we choose a waiting place, and when the painting room is available the model seizes a worker, who paints the clamp, and releases the worker when the task is done.

2.11. Resource Task Start / End



Figure 2.15.: Resource Task Start



Figure 2.16.: Resource Task End

This is used to create a custom task for an agent in a resource pool, for example, if a worker needs to pick up tools before picking up the package. The **Resource Task Start** would be connected to a "Seize" block in the "preparedUnits" port to occupy the agent. Then the seized unit will do the function. When the task is finished and the unit needs to go back to the resource pool a "Release" block is needed. If the agent has to do a custom task after it has been released a **Resource Task End** can be connected to the "wrapUp" port. However, it is not needed if no custom task has to been done after the release.

2.12. Time Measure Start / End



Figure 2.17.: Time Measure Start



Figure 2.18.: Time Measure End

To measure how much time an agent is using in a part of the model the time measurement blocks are used. This can be used to measure lead times and identify bottlenecks. In the Job Shop model, these measurements are implemented multiple times, and the values are used for the statistical calculations. When creating the model, time measurements can be implemented at the end of the model building, when you need to look at outputs of the model. To get the output of the time measurements you can write different commands depending on what you want to do, to get the mean:

```
NameOfTheTimeMeasureEnd.dataset.getYMean()
```

If you just want the value for a given entity you can use:

```
NameOfTheTimeMeasureEnd.dataset.getY()
```

How this can be used in **output analysis** will be explained in the later section.

2.13. Restricted Area Start / End

A restricted area is used to limit the number of agents in a certain area in the model. This area can only have one entry, which is done by using the **Restricted Area Start block**. The restricted area can have more ends if needed, and these are done by using the **Restricted Area End block**.



Figure 2.19.: Restricted Area Start



Figure 2.20.: Restricted Area End

In the Job Shop example, a restricted area is marked in the shipping part of the model. This is done because there is only room for one shipping truck at a time, and if there is no room in the area the truck is parked in a queue.

2.14. Pickup / Dropoff

The **Pickup** block removes agents from a "Queue" block, which is connected to the "inPickup" port. This means that when an agent arrives in the "in" port the agents in the queue (e.g. products) get loaded on the agent arriving in the "in" port (e.g. a truck). The **Dropoff** block removes the agents loaded upon the agent arriving at the "in" port and sends the unloaded agents through the "outDropoff" port.



Figure 2.21.: Pickup

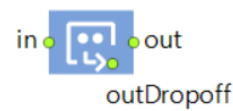


Figure 2.22.: Dropoff

In the Job Shop example, the pickup block is used when the clamps are shipped away on the shipping trucks. Because the capacity of the queue in the shipping area is 5 (TruckCapacity) the number of clamps sent away on the truck is 5.

2.15. Rack Store / Pick



Figure 2.23.: Rack Store



Figure 2.24.: Rack Pick

When using a pallet rack as a storage area in a model, different blocks need to be used. To store entities in the pallet rack a **Rack Store** has to be used. A Rack store moves an agent to the pallet rack. It also seizes a resource from a resource pool that is used to store the agent. The **Rack Pick** is used to remove the entities from the pallet rack and move further into the model. It can also seize a resource from a resource pool to move the entity. This can be seen multiple times used in the Job Shop example to store the pallets and the clamps.

2.16. Select Output

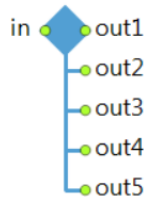


Figure 2.25.: Select output

A **Select Output** decides the route of the incoming agents. The route taken by the agents can be determined in 3 different modes. In the "Probabilities" mode the user can write 5 different probabilities, using a number between 0 and 1. In the "Conditions" mode the user should provide 4 conditions. The conditions work like an if-else-then statement. If the first condition is not true then it moves on to the next. If all 4 conditions are false the agent will exit at "out5". In the "Exit Number" mode the port the entity exits can be chosen. This is typically done by writing "agent.OutParameter", where "OutParameter" is a number between 1 and 5. Which out port the agent exits is defined by this number.

If the user only needs 2 out ports then use the **SelectOutput** block. If the user needs 3-5 out ports the **SelectOutput5** block is used. Simply write 0 or "false", depending on the mode you use, if you do not want to use all 5 out ports. If more than 5 ports are needed, it is possible to combine the SelectOutput blocks by attaching the "in" port to an "out" port of the other SelectOutput block.

3. Input Modeling

Using the source from the Process Library you can alter the input in your model. There are two main forms of inputs that **AnyLogic** can handle, deterministic (Fixed) and Stochastic (Probability distribution). In this section, it will be explained how to implement these. An explanation of how to use parameters and variables in the agents created in the model will also be explained.

3.1. Deterministic input

The first choice of input is deterministic, which is used when there is not any uncertainty. Looking at the Job Shop case, instead of having a stochastic input of products coming into the system, you might need the system to follow a schedule of ordered products. There are two ways of handling the deterministic input, using a rate or an arrival table database/arrival schedule.

3.1.1. Arrival table / Database

One way to import or create a database (Table) into **AnyLogic** is to go under projects, right-click database, and then press new.

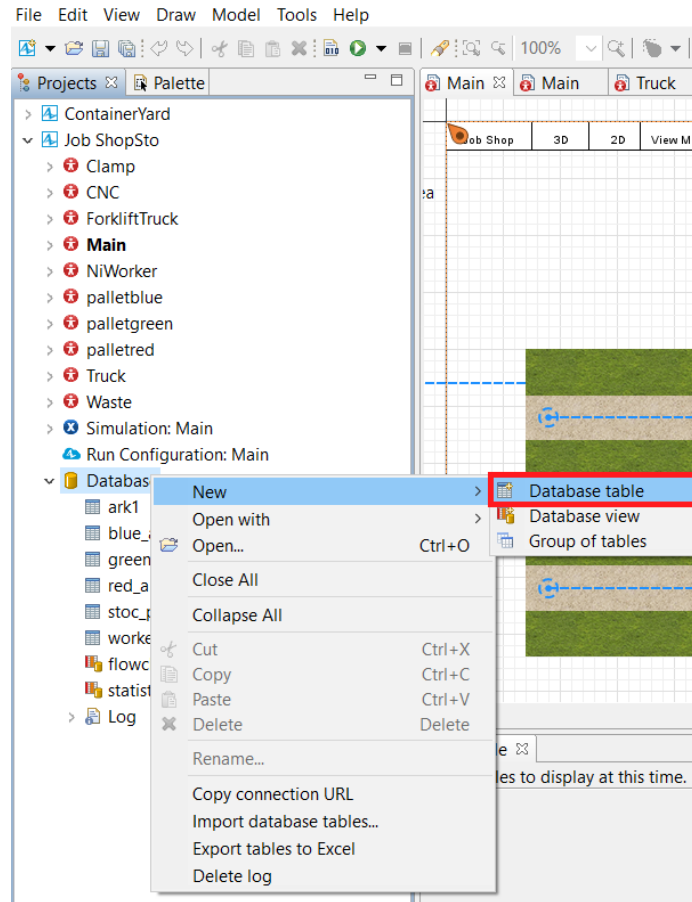


Figure 3.1.: Database creation

In the job shop example, we have imported the arrival schedule of the pallets as *Ark1*, but you can create your own table if an input file is not given. **AnyLogic** databases can handle files from Excel or an SQL Query. This is what a database could look like:

	db_column	blue_arrival	green_arrival	red_arrival
1	12-04-2018 00:00:00	15	10	5
2	12-04-2018 01:00:00	15	10	5
3	12-04-2018 02:00:00	15	10	5
4	12-04-2018 03:00:00	15	10	5
5	12-04-2018 04:00:00	15	10	5
6	12-04-2018 05:00:00	15	10	5

Figure 3.2.: Database in **AnyLogic**

When a database has been created, you can choose a source and refer to it. To do this go under your source-block → Under **Arrivals defined by** choose **Arrival table in database** → Under **Arrival**

date choose the column where the time is defined → Check the box **Multiple agents per arrival** and select it to be a table column, and then select the column in which the input value is defined.

Figure 3.3.: Source arrivals defined by a Database

It is important, when working with time, to remember that **AnyLogic** works in real-time. By default, the model time is set to be the time when you created the model. Therefore you have to check if your database date and model date is the same. You can change the model date by clicking **Simulation: Main** and selecting **Model time**.

Repeated arrival tables: In some cases, you have to simulate until you reach a steady state, and therefore input should be repeated. To do this you have to convert your databases into schedules. Under schedules, you select **Intervals (Start, End)** and then load the values from a database, or type the input yourself. This can be seen in the JobShop example uploaded to DTU Inside.

3.2. Stochastic input

In some cases, the input may not be certain, and therefore changing the input to be stochastic is needed. The stochastic input is just a rate that follows a probability distribution. The three most used distributions is explained in this section.

3.2.1. Normal

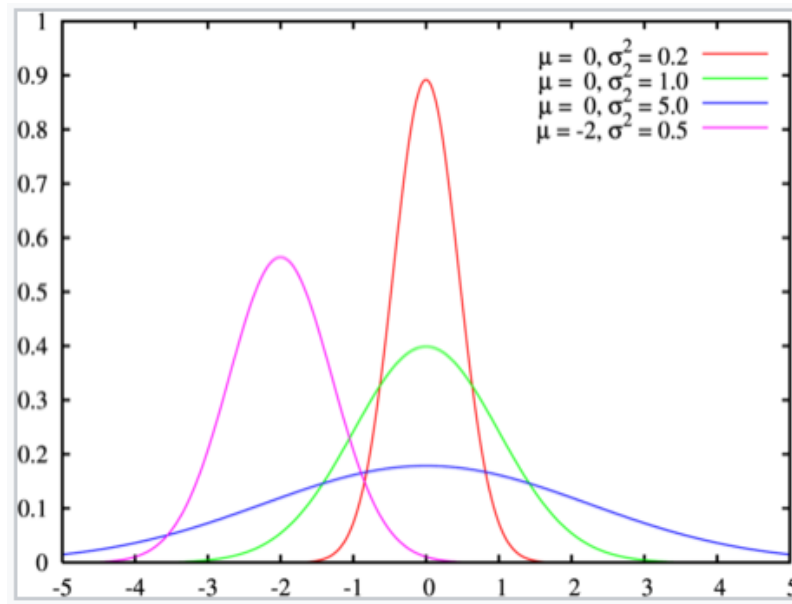


Figure 3.4.: Example of a rate schedule

The normal distribution is the most common distribution, also called the Gaussian distribution. It takes 2 inputs, the mean (μ) and the standard deviation (σ^2). In **AnyLogic** it is written as:

```
normal(sigma^2, mean)
```

The more uncertainty, the higher the standard deviation.

3.2.2. Triangular

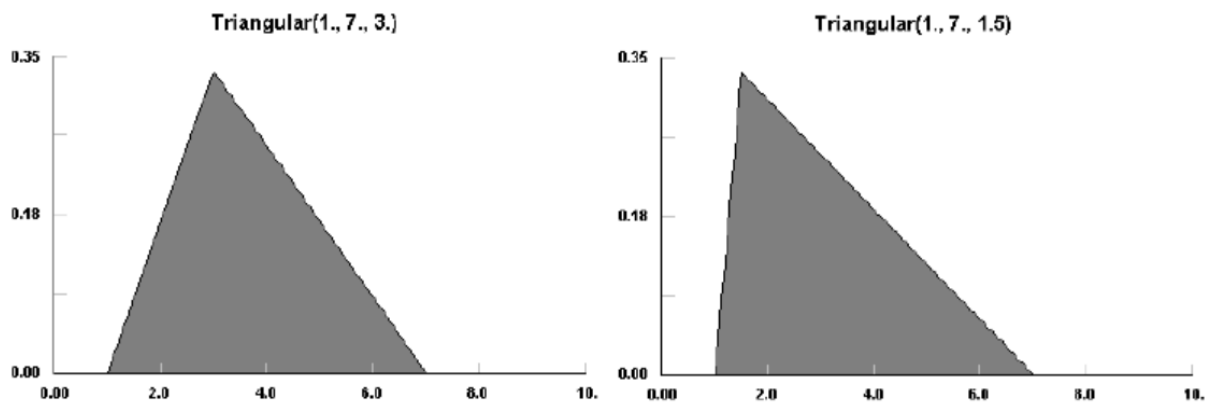


Figure 3.5.: Triangular distributions with different parameters

The triangular distribution is a distribution most commonly used when you do not know much about the true distribution. Therefore you can use the triangular distribution as a subjective description of the population. The triangular distribution takes three inputs, mode, minimum and maximum:

```
triangular(minimum, maximum, mode)
```

3.2.3. Poisson

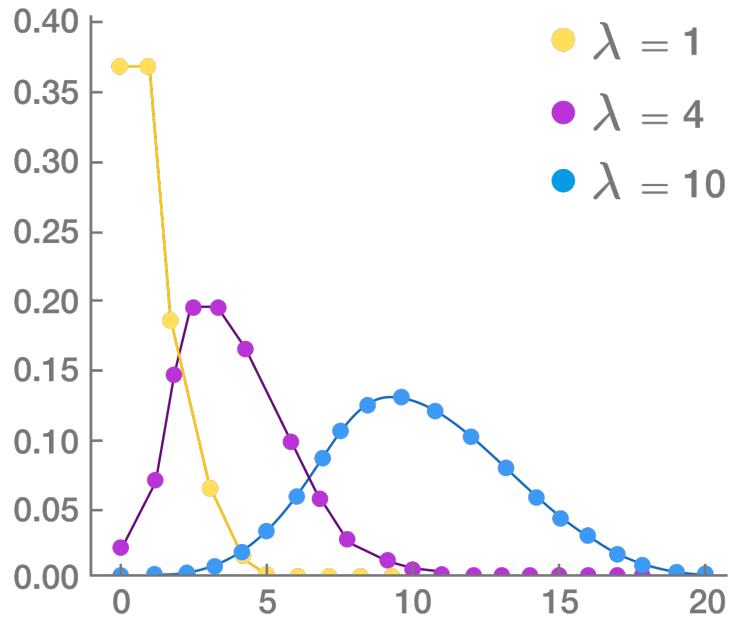


Figure 3.6.: Poisson distributions with different parameters

The Poisson distribution is a distribution explaining the probability of the given number of events happening in a fixed amount of time. For example, how many patients are coming into the Emergency room within the next ten minutes? The Poisson distribution takes one input, rate (λ):

```
poisson(lambda)
```

3.2.4. Rate and Rate Schedule

One way to handle a stochastic input is to create a rate. The simplest way is to change **Arrivals defined by** in a source block to Rate. This rate can be a stochastic distribution, given that the rate parameter is constant over time.

If the rate parameter changes over time a Rate schedule can be created. The process of creating a rate schedule is the same as earlier described in repeated arrival tables. The difference is that "type" changes from **int** to **Rate**.

PalletSchedule - Schedule

Name: ☒ Show name ☐ Ignore

Visible: ☒ yes

Data

Type:

Unit:

The schedule defines: ☐ Intervals (Start, End) ☒ Moments

Duration type: ☐ Week ☒ Days/Weeks ☐ Custom (no calendar mapping)

Repeat every:

☐ Snap to:

☐ Loaded from database

Time	Value
00.00	15.0
01.00	15.0
02.00	15.0
03.00	15.0
04.00	15.0

Figure 3.7.: Example of a rate schedule

In the source block, you can now choose **Rate schedule** in **Arrivals defined by**. The rate used in rate schedules is by default an exponential distribution, but it is possible to alter this. This is done by checking the box **Modify rate**, an example of this could be that the rate varies dependent on a normal distribution.

sourcePalletBlue - Source

Name: ☒ Show name ☐ Ignore

Arrivals defined by:

Rate schedule:

Modify rate: ☒

Rate expression:

Set agent parameters from DB: ☐

Multiple agents per arrival: ☐

Limited number of arrivals: ☐

Figure 3.8.: Example of modifying the baseRate

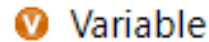
baseRate is the rate called from the **Rate schedule**, and then a variance of 1 is called from the normal distribution.

3.3. Parameters and variables



Parameter

Figure 3.9.: Parameter



Variable

Figure 3.10.: Variable

The parameter and variable blocks can be found under **Palette** → **Agent**. In **AnyLogic** parameters are usually static values that do not change during the simulation. Parameters can be used in different ways. It can be a static value that is defined by the user beforehand (e.g. a time for a process). To assign a value to the parameter the type must be chosen correctly (e.g. double or int) and a default value must be assigned to the parameter (e.g. 3.5 or 3). For calling the parameters value you simply write the name of the parameter. If a parameter is calculated based on other parameters (e.g. by writing a function in the default value box), the parameter has to be changed from a static to a dynamic parameter. This is done in the Advanced panel under Properties. To call the value of a dynamic parameter you have to write the name followed by round brackets (parameterName()).

If the parameter value change during the time a simulation is running, a variable should be used instead of a parameter.

Parameters and variables can also be used as values for the agents created in the model. To assign parameters and variables to the agents you can simply drag the parameter or variable into the agent. This is done by opening your agent under **Projects** and the model you are using. When you open the agent it will most like be empty except for the picture you assigned to the agent. Now the parameter or variable can be dragged into the agent.

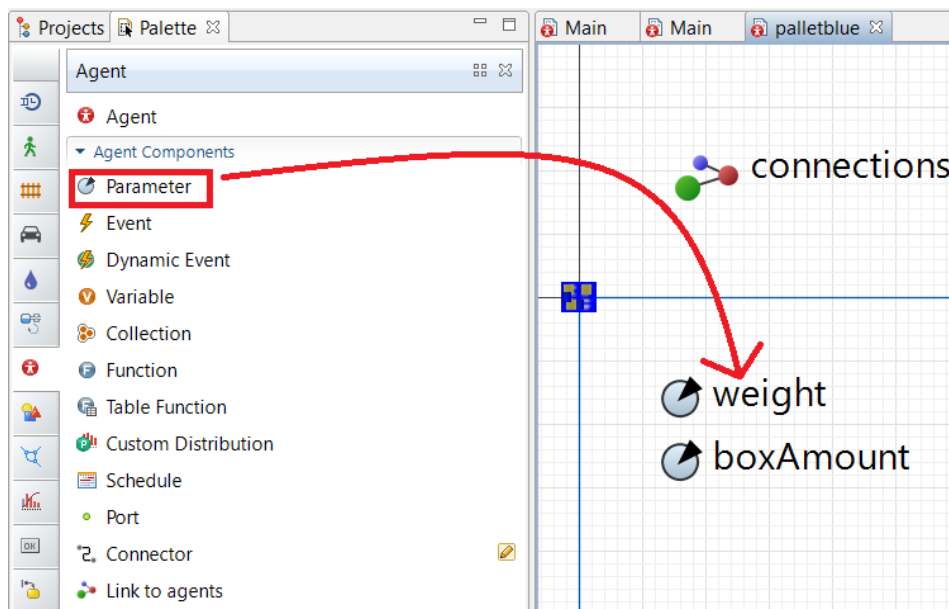


Figure 3.11.: Example of parameters within an agent

Above is an example of how a pallet agent has 2 parameters (weight, boxAmount). Now these parameters can be used within your model. To use the parameter or variable in the model you can write agent.parameterName (e.g. agent.weight) or agent.variableName.

A variable can be used to calculate a value of an entity during the system and is found just below the parameter (see Figure 37). This can be useful in a system that needs to act differently according to different scenarios. For example, if a pallet has been in the rack for too long it is discarded. How to define this variable can be seen below:

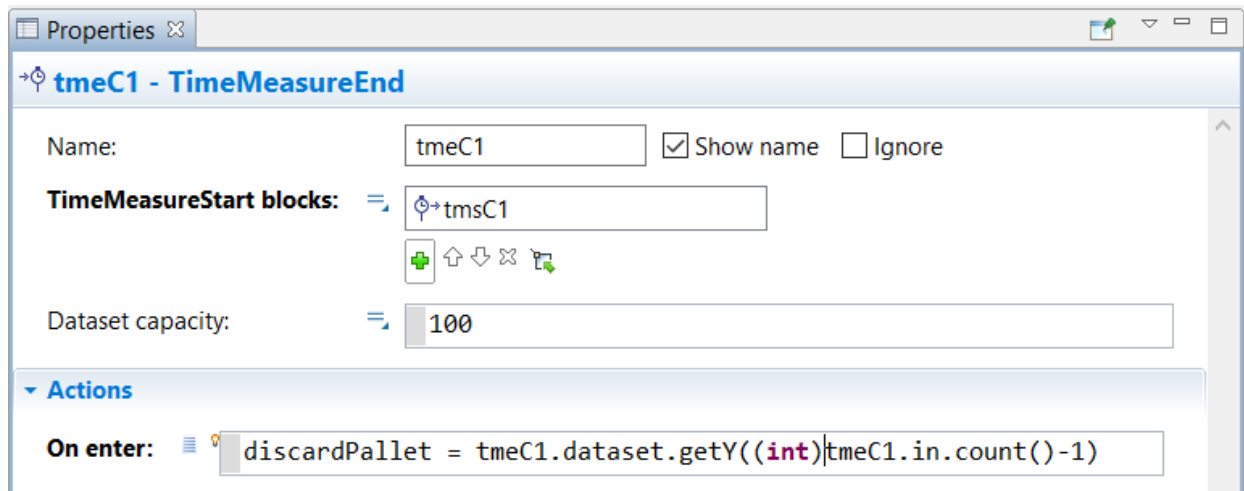


Figure 3.12.: Example of a variable calculated in a TimeMeasureEnd

Notice that the variable is called **discardPallet** and the value changes every time an entity enters **tmeC1**.

3.3.1. Parameters from databases

When simulating the parameters of the agents in your model might have to change depending on a value given in an Excel sheet. An example of this could be when you consider the pallets arriving in the Job Shop you already know the weight of the pallet and the number of boxes on the pallet, and these values will change depending on the different pallets entering the system. This data can be written in an Excel sheet and can be used as the values of the parameter. You can assign these values to the parameter in the source block of the pallet as shown below:

Properties

sourceBlueParameters - Source

Name: ☒ Show name ☐ Ignore

Arrivals defined by:

Database table:

Arrival date:

Multiple agents per arrival: ☐

New agent:

Agent parameters mapping:

Parameter	Column
weight	weight
boxAmount	boxamount

Location of arrival:

Figure 3.13.: Examples of parameters within an agent

In the example above you can see that the parameter `weight` gets the values that are written in the column with the name `weight` in the Excel sheet. To do this the Excel sheet has to be imported into [AnyLogic](#) described in section 3.1.

4. AnyLogic tools, blocks and functions for Output Analysis

When the model is finished, it is relevant to see the statistical output and analyze how the model performed. To get any output from **AnyLogic** it is necessary to tell the model to log the execution. To log the database you need to look at **Projects** and your model. Here you need to click on **Database** and click on "Log model execution" under **Properties**.

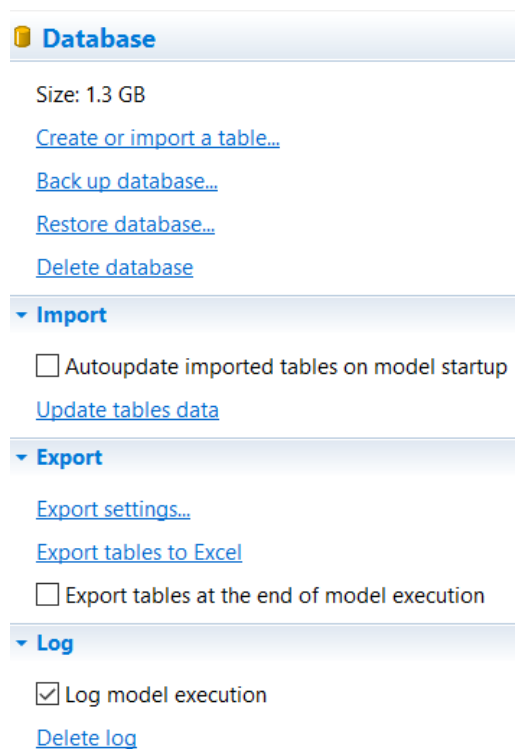


Figure 4.1.: Database properties

Under the **Properties** of the database it is also possible to export the output to Excel. This is done by clicking on "**Export settings...**". Here you can choose which tables you want to export (do not export it all as it is a big database) as shown below:

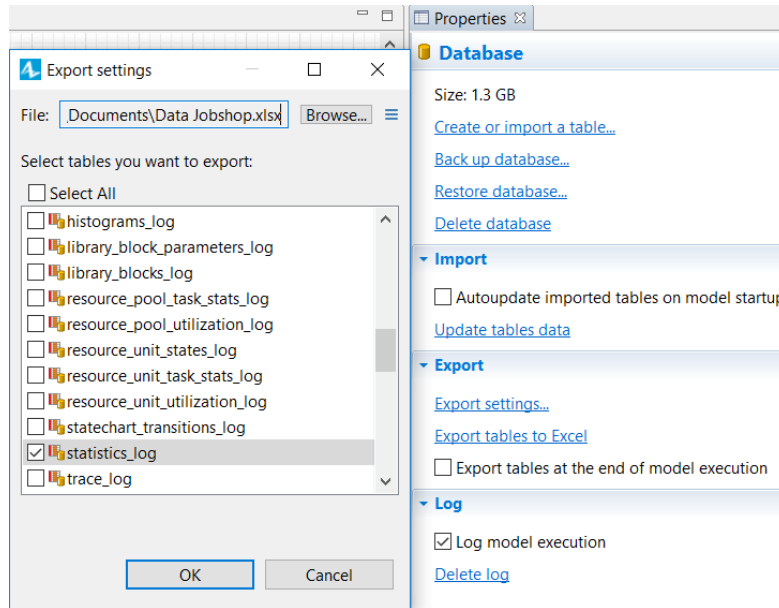


Figure 4.2.: Database to Excel

When the wanted tables have been chosen you click **"OK"** and then click on **"Export tables to Excel"**. Some of the tables contain information that is hard to use as a meaningful output. To get an overview of what the different tables contain go through them all and look. Two tables that contain useful information are the **"statistics_log"** and the **"flowchart_stats_time_in_state_log"**. If the time measurement blocks have been used (which they should have), then information regarding the time blocks will be logged in **"statistics_log"**. Here it is possible to see the meantime (and more) used within the **Time Measure Start** and **Time Measure End**.

Under **"flowchart_stats_time_in_state_log"** information about the different activity types of the block can be found (e.g. if they are working or waiting).

These datasets usually only make sense to look at when running single simulations.

4.1. Statistical calculations

4.1.1. Output block

Under the palette **Analysis** the output block can be found. The output block can be used to calculate static calculations. This is very useful when running multiple replications (see section 5.3). By default the output block calculates a value at the simulation end. But the block can also update continuously during model time, or at a specified time. It can also be calculated with user control, where the user has to call the function **update_nameOfOutput()** in a block to update the output's value. The output block only gives one value.

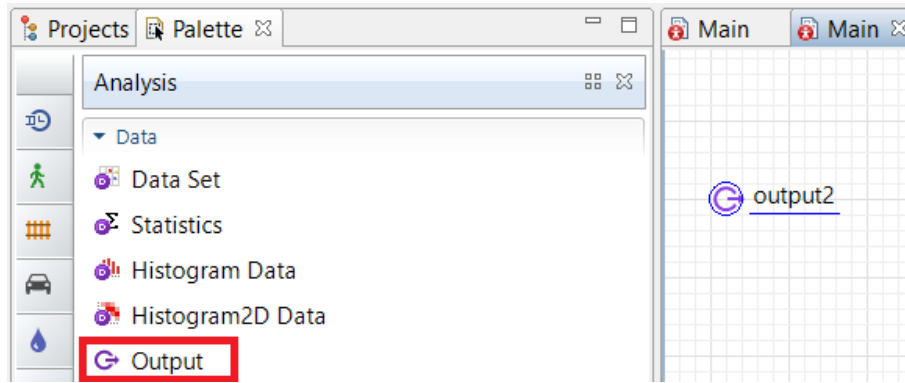


Figure 4.3.: Output block

An example of this could be the mean lead time of every simulation we run.

4.1.2. Data set

When creating calculations it can be beneficial to log data that **AnyLogic** does not log automatically. This can be done by selecting the Analysis palette, and choose **Data Set**.

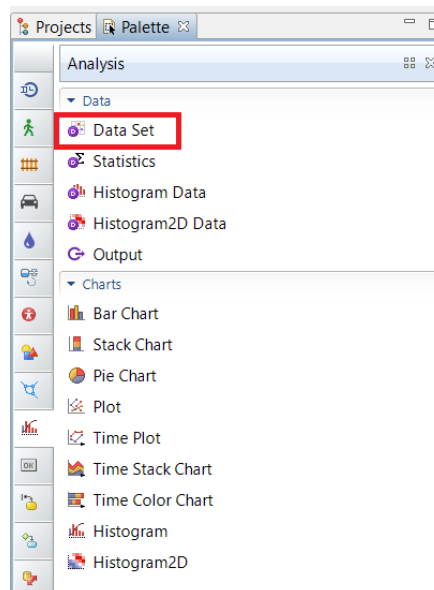


Figure 4.4.: Analysis Palette

The **Data Set** function has multiple features, the most common is to log some kind of data over time. If this is the case check the box **Use time as horizontal axis**. When creating the **Data Set** you need to specify what values that has to be logged, how much data you want to store, and if the **Data Set** needs to update automatically. Usually, if you use time as the horizontal axis you want to keep as many samples as the runtime of the simulation, meaning if you run the simulation for 1000 minutes, and the dataset updates every minute, you should keep 1000 samples.

ResourceCost - Data Set

Name: ☒ Show name ☐ Ignore

Visible: ☒ yes

☒ Use time as horizontal axis value

Horizontal axis value:

Vertical axis value:

Keep up to latest samples

☒ Update data automatically
☐ Do not update data automatically

☒ Use model time ☐ Use calendar dates

First update time:

Update date:

Recurrence time:

☒ Log to database

[Description](#)

Figure 4.5.: Data Set Properties

This data will be logged in the **datasets** log. Where the X-value is the continuous time variable, and the Y-value is the cost at that minute.

	agent_type	agent	name	index	x	y
3001	Main	root	ResourceCost	0	1	30
3002	Main	root	ResourceCost	1	2	30
3003	Main	root	ResourceCost	2	3	30
3004	Main	root	ResourceCost	3	4	21
3005	Main	root	ResourceCost	4	5	9

Figure 4.6.: Datasets log

In the Job Shop example, it could be relevant to look at the cost of the production over time. In the **Vertical axis value** the cost function is defined. In the example one of the cost functions is defined as this:

```
forklifts.busy()*ForkliftCost+cnc.busy()*CNCost+workers.busy()*WorkerCost
```

This functions states every minute a forklift from the resource pool is working (**busy()**) a cost parameter is multiplied. And the same for the resource pool of cnc and workers.

4.1.3. Excel files

It is possible to get **AnyLogic** to write out Excel files with a chosen output. This can be useful in multiple scenarios, for example, if you want all the time measures of entities in the system. You need to create an Excel file from the **Connectivity** palette and select an Excel file on your computer in

which you want to write data. **AnyLogic** does not create an Excel file for you, so you have to create your own.



Figure 4.7.: Excel File

Then you need to call a function in a block (e.g. `TimeMeasurementEnd`). One **AnyLogic** function to do this is `setCellValue`, and is written as:

```
excelFile.setCellValue(value, sheet, row, column);
```

- value: the value you want to save in the Excel sheet. This can also be a variable, to save that you simply write the variable name. In the case of text write it between air quotes, like a string (e.g. "column name").
- sheet: the number or the name of the sheet (in case of a name write it like a string, e.g. "Ark1")
- row: the number of the row
- column: the number of the column

This will write one value, but it is possible to index it to get multiple values. An example of this can be seen in the Job Shop example:

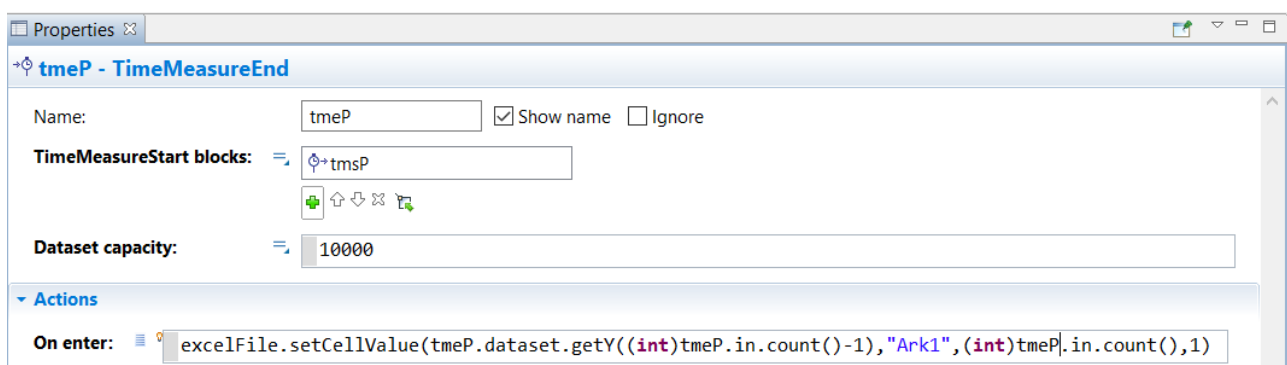


Figure 4.8.: Inserting values into an Excel file

The wanted output is the time of all pallets that have been in the painting room, and the row changes every time a new pallet comes in. The index in the output has -1 because Java indexes with the start of 0.

4.1.4. Functions

busy() is a function created by **AnyLogic** that tracks if a resource pool is active. But not all needed functions are defined in **AnyLogic** and therefore need to be programmed. The first step is to create a function block from the **Agent palette**

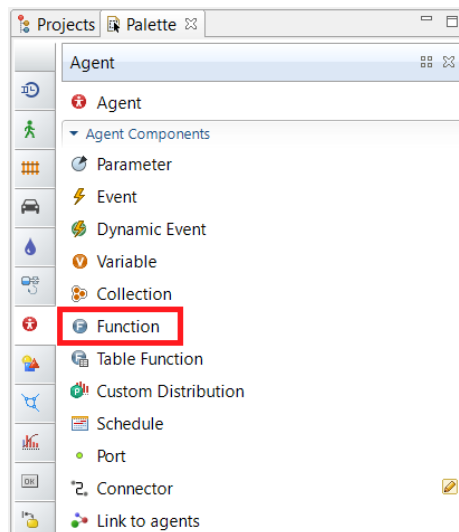


Figure 4.9.: Agent Palette

Inside this block, you can define every needed step. Recall the sum-function explained in section 2.2 **For-loop**, if this is implemented into **AnyLogic** it will look like:

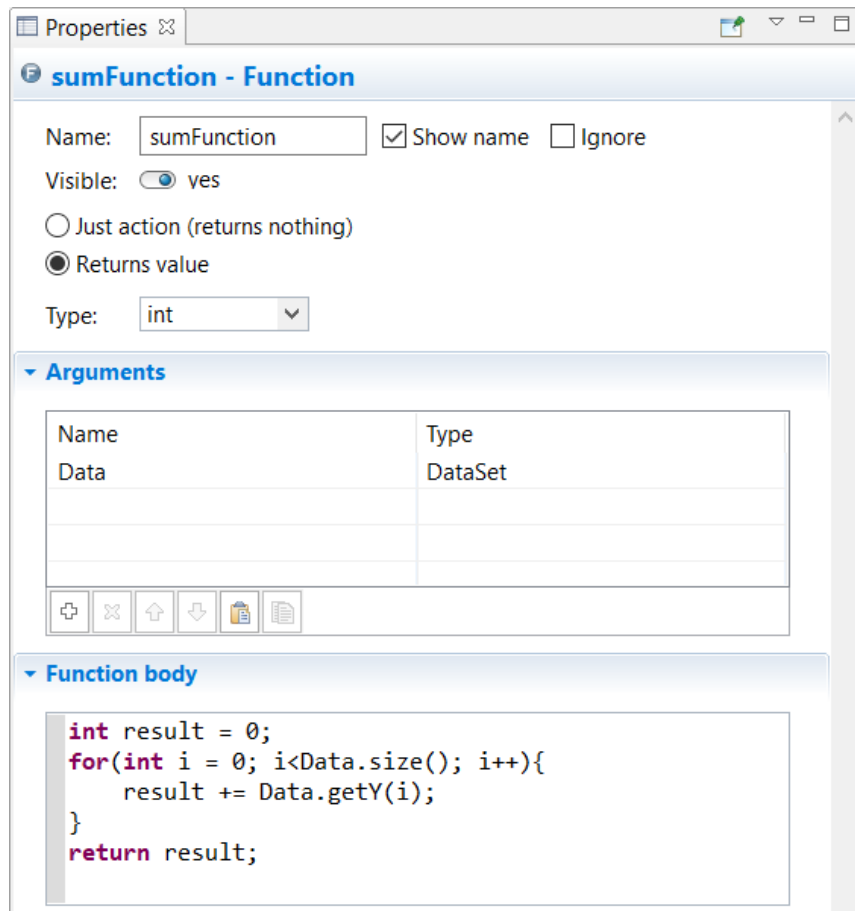


Figure 4.10.: Sum function

From the function we need to specify what it returns (in this case a value), what type of value that is (in this case an integer), and how many arguments we use in the function and what type they are (in this case Data, which is a DataSet). The Function body specifies how the function operates, in this case it is the exact same code in section **2.2 For-loop**. In the Job Shop example, this is used to calculate the cost over time from the Data Set ResourceCost. The ResourceCost data set explained in the earlier section is the cost at a given time in the simulation, therefore the total cost at a given time would be this data set summed through all indexes. Using the sumFunction on ResourceCost would look like this:

```
sumFunction(ResourceCost)
```

This can be seen in the **Profit over time** plot and will be explained in the next section.

4.2. Visualizations

Visualization of data values is vital when the user has to get the right perception of the simulation. Therefore making good visualizations are very important. When all the values are created the user can just export the data into Excel and create the needed visualizations. However, creating the plots in **AnyLogic** will give a quicker response when changing e.g. parameters. In the Job Shop example,

it could be relevant to simulate how big certain resource pools need to be. When the plots are not created in **AnyLogic** the data would have to be exported for every changed parameter.

In this section the basic plots are explained, and how they are done in the Job Shop example. Under the Analysis Palette, you will find all the plots for the visualizations.

4.2.1. Time plot

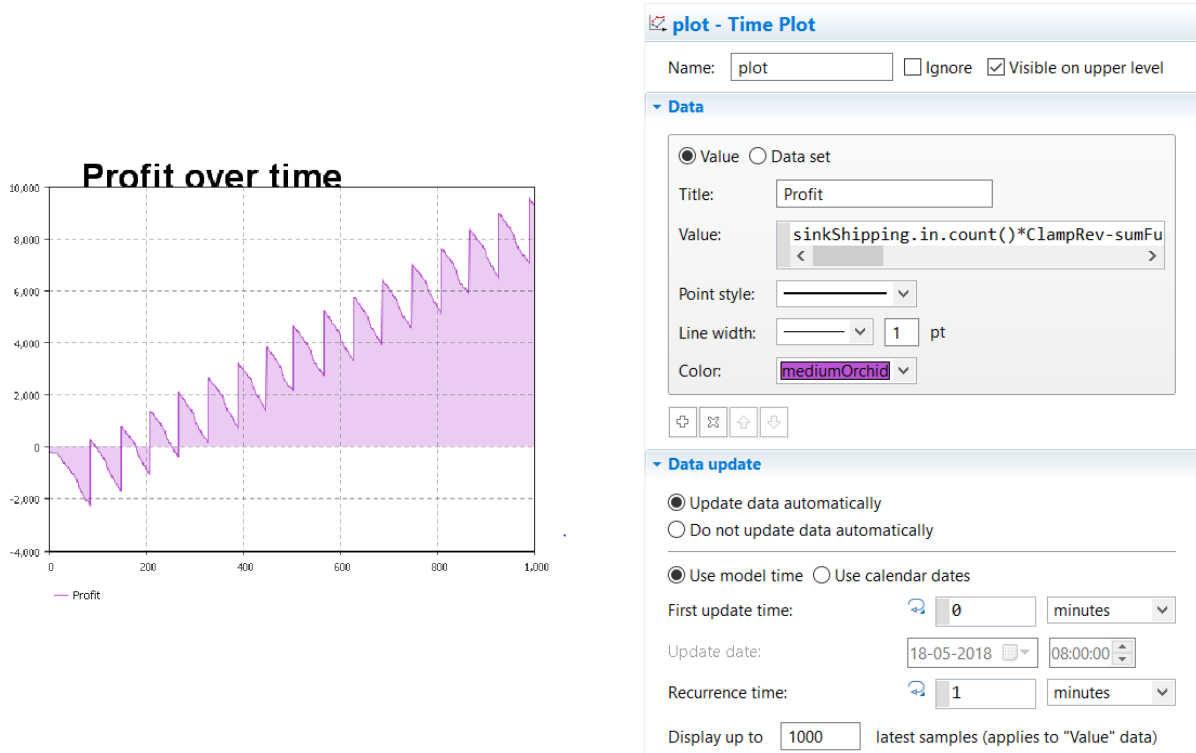


Figure 4.11.: Time Plot

Time plots are good at showing how certain values act over time. In the plot shown above you can see the time on the x-axis and the calculated value on the y-axis. In the Job Shop example, the value that is calculated is the profit over time. The revenue is the number of clamps leaving the job shop, and the cost of running the job shop is subtracted from this to calculate the profit. This is done using the code below:

```
sinkShipping.in.count()*ClampRev-sumFunction(ResourceCost)
- (sourcePalletRed.out.count()*CostRed + sourcePalletGreen.out.count()*CostGreen
+ sourcePalletBlue.out.count()*CostBlue)
```

Each pallet costs a certain amount (e.g. `sourcePalletRed.out.count()*CostRed`), while the cost of running the machines is done by using the sum function on the data set `ResourceCost` mentioned earlier.

4.2.2. Bar Plot

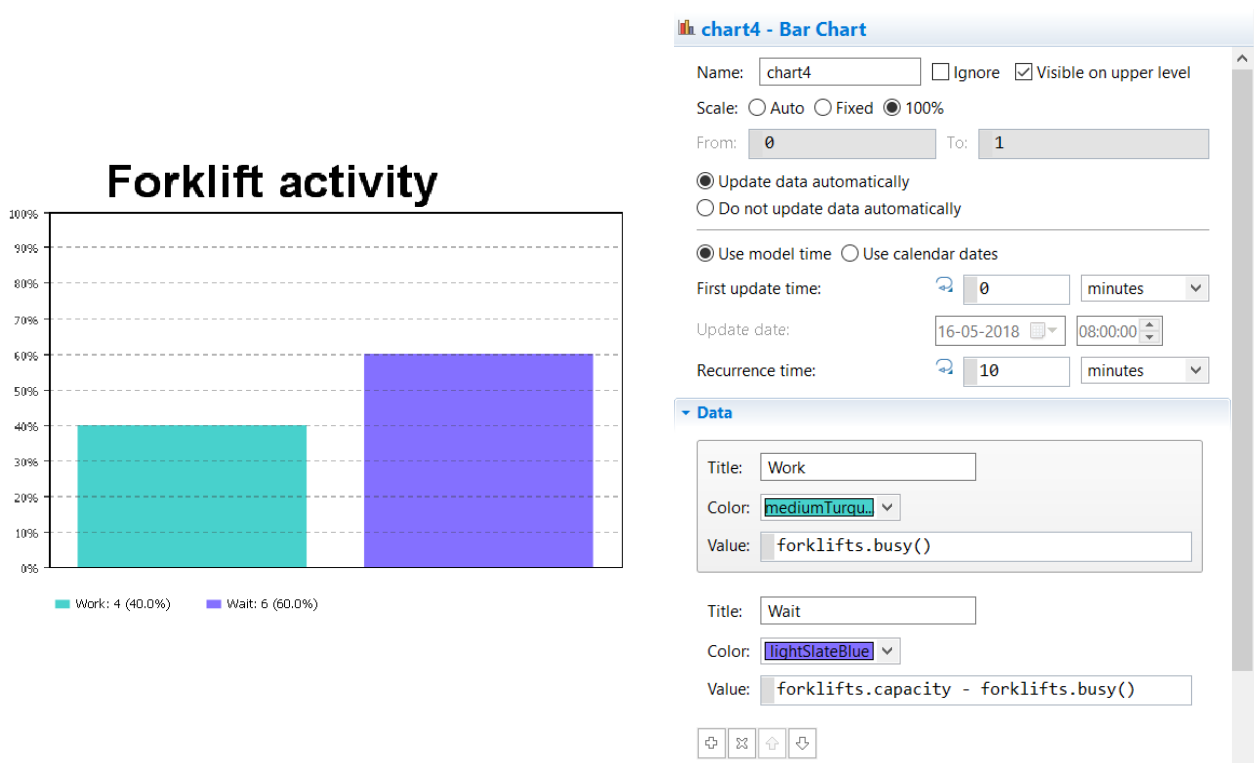


Figure 4.12.: Time Plot

Bar plots are great when comparing measurements. In the Job Shop example it is interesting to see how many forklifts are working compared to how many there are not.

Stack Chart Plot

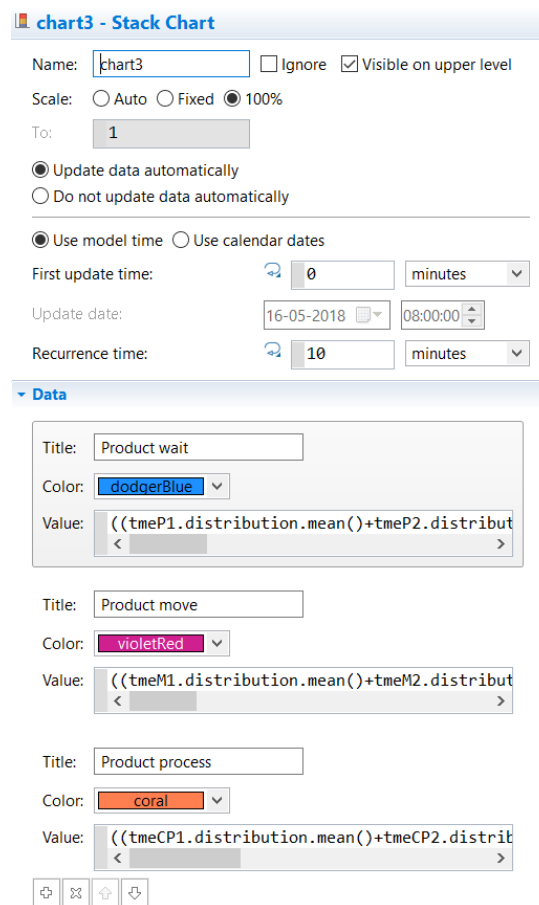
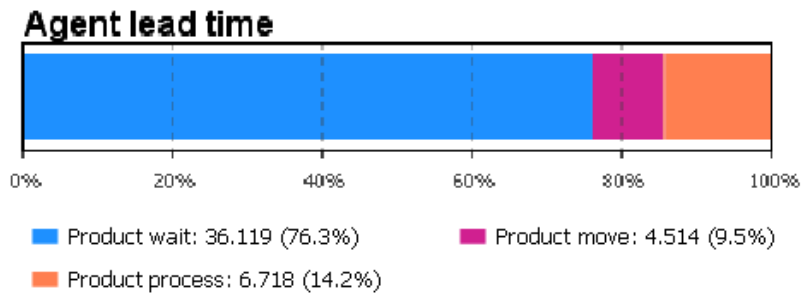


Figure 4.13.: Time Plot

Stack charts can both be Bar charts and Pie charts, they are useful when comparing proportions (Percentages). The stacked bar chart was used to analyze bottlenecks in the Job Shop example. From the plot above we see that the agents spend above 75% of the time waiting. Therefore a big bottleneck must be in the system somewhere. Plotting the proportions of the waiting time, it can be seen that the agents wait before the CNC-machine process. From that, a hypothesis can be created that implementing another CNC machine in the system will decrease the lead time. This can therefore be tested by changing the number of CNC machines and simulate the model again.

4.3. Analyzing output

As referred to in Chapter 7 of "Simulation Modeling and Arena" by Rossetti (2016), the inputs in a simulation model are random; hence, the outputs are also random. In **AnyLogic**, you can control the randomness through the dedicated pane in the simulation experiments.

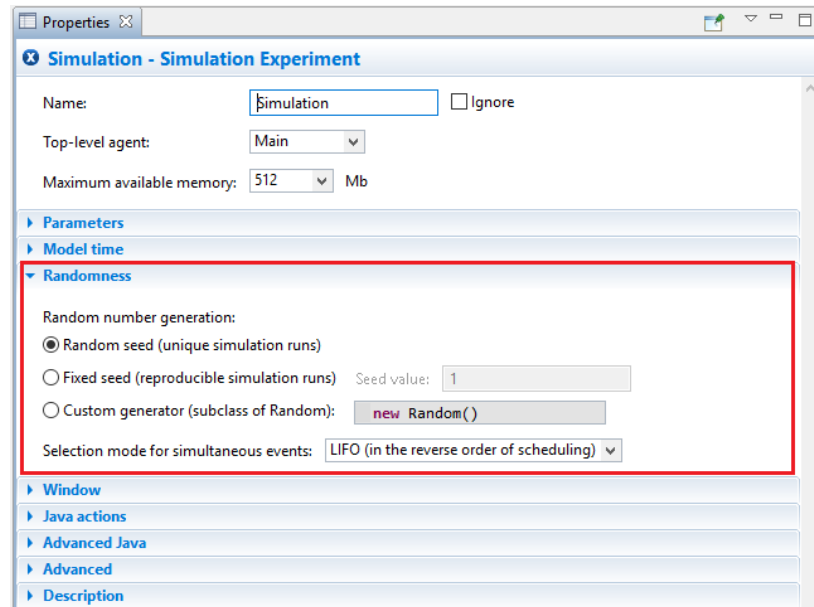


Figure 4.14.: Randomness in the simulation experiment

There are three possibilities for modeling the randomness in **AnyLogic** :

1. Random seed: the software initializes a different seed for each experiment performed. Hence, the model runs cannot be reproducible.
2. Fixed seed: the user sets a fixed seed for the randomness. This option is very valuable in the development phase as the simulations are reproducible.
3. Custom generator: **AnyLogic** also permits the user to create his own random class.

Performing the output analysis, a simulation experiment aims to observe a set of outputs (over time or at the end of the experiments) based on the input parameter values. If your model is stochastic, the results of the single model run may not be representative and they change over the random number. Therefore, more replications with an independent random numbers should be run in order to make valid conclusions.

AnyLogic affords an opportunity to run the model with different model parameters and analyze how certain parameters affect the model behavior. You do not need to run your model several times one by one and change parameter values manually after each model run, trying to remember the results of these runs and compare them. Using the "Parameter Variation" experiment you can configure the complex model simulation comprising several single model runs, varying one or more root object parameters. Running this experiment with fixed parameter values you can also assess the effect of random factors in stochastic models.

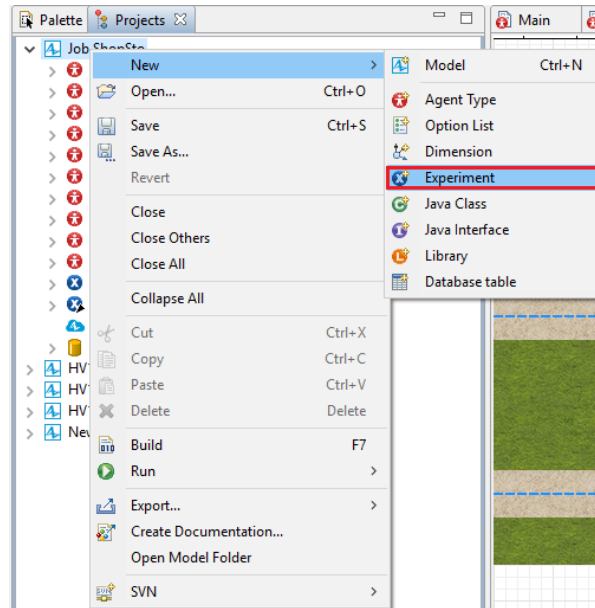


Figure 4.15.: How to create a parameter variations experiment

To add a "Parameter Variation" experiment to your model: right click on the model → New → Experiment. Then, from the experiment type choose Parameter Variation.

In the Properties pane, you can set up the parameters for the variation experiment:

Parameters: ☒ Varied in range ☐ Freeform

Number of runs: 10

Parameter	Type	Value		
		Min	Max	Step
TruckCapacity	Fixed	5		
RedAss...lerCap	Fixed	1		
GreenA...lerCap	Fixed	2		
BlueAss...lerCap	Fixed	3		
TruckSpeed	Fixed	40 //KM per hour		
RackLiftTime	Fixed	30 //Seconds		
RawMat...alMean	Fixed	20		
RawMaterialMax	Fixed	25		
RawMaterialMin	Fixed	15		
ProcessCNCMin	Fixed	2		
Proces...CMean	Fixed	3		
ProcessCNCMax	Fixed	4		
ProcessAssMin	Fixed	1		
ProcessAssMean	Fixed	2		
ProcessAssMax	Fixed	3		
PaintMin	Fixed			
PaintMean	Fixed	2		
PaintMax	Fixed	3		
ForkliftCost	Fixed	3		
CNCCost	Fixed	10		
WorkerCost	Fixed	2		
CostRed	Fixed	6		
CostGreen	Fixed	4		
CostBlue	Fixed	2		
ClampRev	Fixed	2500		

Figure 4.16.: Parameters' pane from the Job Shop example

- Varied in range: you can define the range of variation of the parameters in your model. Therefore, every combination of parameters constitutes an experiment. To do this you create a parameter in the main and refer to it, e.g. the capacity of the worker resource pool. This will make it possible to vary the number of workers available. Then you change fixed parameters to a range and specify the min, max and step it should vary in.
- Freeform: model is run for the fixed number of iterations you specify. Parameter values are calculated according to the expressions you define.

After choosing your model time and randomness type, the following pane is dedicated to the replications.

Figure 4.17.: Replications' pane

- Fixed number of replications: you explicitly define the number of replications to be run for every iteration.
- Varying number of replications: you will specify the minimum and a maximum number of replications; then, the replications stop evaluating a solution when:
 - the confidence interval is small enough to fit in the given percentage of expression value, or
 - current expression value is not inside the mean confidence interval (not near the mean value), or
 - the maximum number of replications run.

Parameter variation can be seen done in the Job Shop example with 100 replications. The Job Shop example also shows how to create output for the different simulations. In advance of the parameter variation, it is important to check off the box **Allow parallel evaluation**.

5. Output Analysis using replications

5.1. Using replications will increase the credibility of a simulated output

After Phase I and II of the general simulation methodology focused on problem definition and model building, managers are eager to obtain concrete results and insights from their simulation model. However, it is in this situation that you should be very careful to not move forward too quickly, because, without careful consideration of the obtained results, the risk of making decisions on wrongful insights is highly present.

To elaborate on this, it is relevant to include a statement from Rossetti stating that *"random input implies random output"* (Rossetti 2016). To understand this, you can consider the underlying concept of a simulation model, because basically, a simulation model is like a mathematical function with specific input parameters that will lead to a given output result. However, on the contrary to a mathematical function like $x^2 + y^2$ with $x = 3$ and $y = 5$ that will always be 34, in a simulation model there is not the same certainty, because one or more of the input parameters follow a random distribution. When input parameters are random, your obtained output will also have a degree of randomness associated with its value. In another way, you can say that your output value(s) is a combination of the actual performance of the system and a degree of *noise*, stemming from the randomness of your system, that affects the observed performance in a positive or negative direction to a degree that is unknown.

5.1.1. Random noise in the modified Job Shop tutorial

To practically illustrate the random noise's influence on an output KPI, the Job Shop model from Anylogic (AnyLogic n.d.) can be utilised. Compared to AnyLogic's tutorial model, an additional paint and related drying process have been added after the CNC-process. For clarification, please find the model's activity diagram in Appendix B.

Within this model, multiple input parameters exist. Of these, the number of CNC machines, forklifts and workers are explicitly defined while all processing times are defined to follow specific random distributions as seen in Appendix B. Therefore, when simulating the model, the random noise will affect the output result to an unknown degree. This can be seen when plotting the KPI named *Average time in system*, which is a measure of the average time it takes for raw material to be picked up from the delivery truck until its related finished product is placed on the outgoing shipment truck. In figure 5.1, this KPI is plotted across 100 independent, but in terms of input parameters identical, simulation runs of 24 hours.

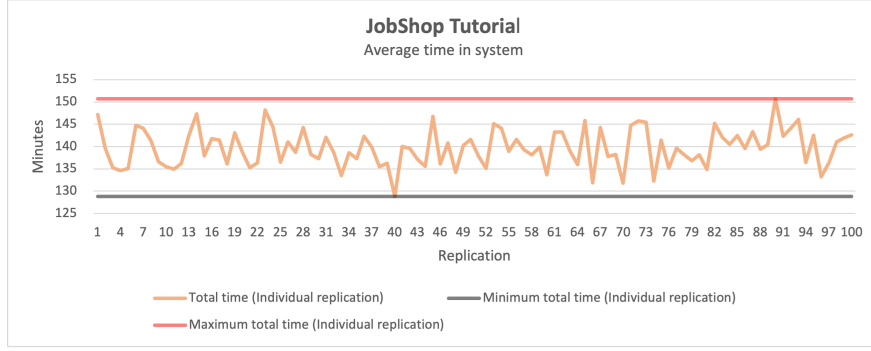


Figure 5.1.: Graph illustrating the KPI across 100 independent runs

Through the graph, the varying degree of random noise becomes clear with the KPI value ranging from 129 to 151 minutes. This is a deviation of $\pm 8\%$ compared to the average KPI value of 140 minutes across the 100 independent simulations. This simple illustration therefore clearly outlines that addressing the randomness of one's output is key to establishing the best possible foundation for decision-making. Because if only one experimental run is simulated and it is assumed that this single run's output result(s) is representative, you risk making your decisions on a KPI that, in this case of the modified Job Shop model, is 8% away from actual performance, which most likely will lead to wrongful decisions. To counter this risk, the use of repetitive replications and across-replication statistics becomes beneficial.

5.1.2. True performance of the modified Job Shop tutorial

Multiple replications and reporting your output result(s) as across replication statistics will allow you to directly address the noise from the randomness and by that express the actual performance of your system. This ability is visualised in figure 5.2 where figure 5.1 is supplemented by the continuous across replication mean of the KPI. This value is calculated for each replication $i = 1 \dots 100$ by equation (eq.) 5.1.

$$\bar{X}_i = \frac{\sum_{j=1}^i X_j}{i} \quad (5.1)$$

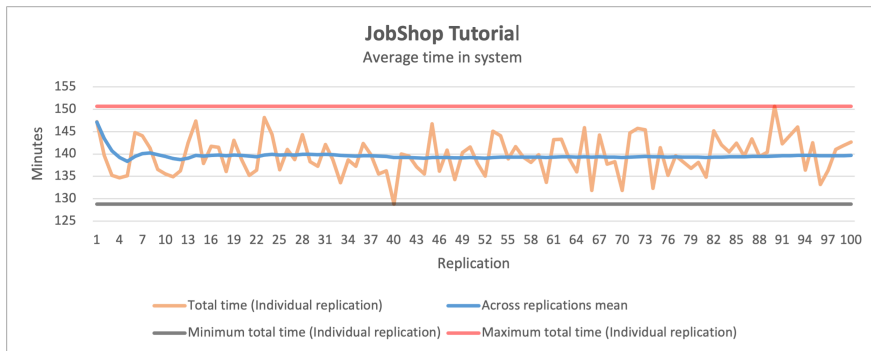


Figure 5.2.: Graph illustrating the across-replication mean of the KPI across 100 independent runs

Through figure 5.2, the benefit of multiple replications and using across replication KPIs for decision-making is visualised, because the randomness is addressed to instead visually show the actual system

performance. By therefore conducting multiple replications and using the across replication mean of the KPI, it will be possible to make confident decisions based on the true performance of the system and not its related randomness.

5.2. Required number of replications

To ensure confidence in your decision-making, it is visualised that multiple replications and across-replication statistics are required. However, a concrete number of required replications does not exist and the question is therefore how one can ensure the needed confidence for decision-making. For this task, three different approaches described by Rossetti (2016) and Sheldon Ross (2013) will be explained.

But first, it is important to define what is meant by the needed confidence for decision-making. To explain this, the Job Shop KPI from sec. 5.1 can be used. Across 100 independent simulation runs, this KPI varied between 129 and 151 minutes and due to this level of uncertainty, the Job Shop manager wants to be 95% confident that the final KPI value reported in the production run of the simulation experiment's phase three and four is within one minute of the true performance of the system.

In this example, the requirement of $\pm$ one minute is called the required half-width, h , and the 95% threshold is called the confidence level, which is naturally connected with the risk, α , that in this case would be 5%. Therefore what the three approaches by Rossetti (2016) and Ross (2013) in this example would define is the required number of independent and in terms of input parameters identical replications, n , to ensure only a 5% risk that future decisions will be based on an across replications mean that is more than one minute away from the true system performance.

5.2.1. Half-width method by Rossetti

The first approach available is the half-width method described by Rossetti (2016), which is based on the normal distribution and the related t-distribution. When the value distribution of a random variable, like the Job Shop KPI, follows a normal distribution, the t-distribution can be used to define the required confidence in the reported mean value across independent samples of such KPI.

In simulation, the randomness affecting one's system is assumed to be normally distributed and this therefore allows Rossetti to utilise the ability of the t-distribution to calculate the required number of replications, n . To practically illustrate that this assumption is acceptable, the value distribution of the Job Shop KPI in figure 5.1 is illustrated in figure 5.3. Through this practical visualisation, it becomes clear that the distribution of the 100 independent samples of the KPI has a high similarity to the normal distribution practically illustrating the acceptance of this assumption.

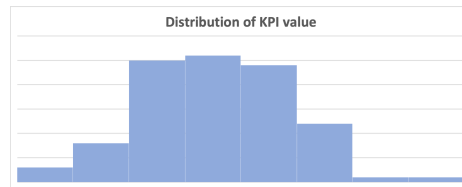


Figure 5.3.: Distribution of KPI value in figure 5.1

To calculate the required number of replications, n , Rossetti's (2016) method therefore first requires a pilot experiment of e.g. 10, 15 or 20 replications to get an estimator for the std. deviation experienced

in the value distribution of the KPI. With this estimate, the half-width experienced in the pilot experiment, h_0 , can be calculated through eq. 5.2 where n_0 is the number of replications in the pilot experiment, s is the observed std. deviation and α is the accepted risk of e.g. 5% as was the acceptable risk for the Job Shop manager in the explanatory example.

$$h_0 = t_{\alpha/2, n_0-1} \frac{s}{\sqrt{n_0}} \quad (5.2)$$

With this initial half-width, the required number of replications, n , to ensure the desired confidence can be calculated through eq. 5.3 where h is the defined half-width similar to the +/- one minute required by the Job Shop manager in the explanatory example. In this regard, it should be noticed that the unit of the half-width always follows the unit of the KPI in question.

$$n \cong n_0 \left(\frac{h_0}{h} \right)^2 \quad (5.3)$$

To facilitate the practical usage of the described formulas defined in eq. 5.2 and eq. 5.3, the following steps will allow you to to utilise Rossetti's half-width method on a selected KPI:

1. Conduct a pilot experiment of the model with an initial number of replications $n_0 = 15$
2. Calculate the KPI's average across the n_0 replications
When conducting the analysis in Excel, the function **AVERAGE** is used
3. Calculate the std. deviation, s , for the KPI in the pilot experiment through $s = \sqrt{\frac{1}{n_0-1} \sum_{i=1}^{n_0} (x_i - \bar{x})^2}$
When conducting the analysis in Excel, the function **STDEVA** is used
4. Define your required confidence interval and related α
5. Calculated h_0 through eq. 5.2
When conducting the analysis in Excel, use **T.INV** to obtain the correct value from the t-distribution and remember to use $(1 - \frac{\alpha}{2}; n_0 - 1)$ as input to the **T.INV** function
6. Define your desired half-width h
7. Calculate the required number of replications n through eq. 5.3

5.2.2. Iterative process by Ross

The second available approach is an iterative process introduced by Ross (2013), which is focused on limiting the std. deviation observed in an extensive pilot experiment. It is because Ross argues that when a large number of observations are used, the likelihood of a random variable being too many std. deviations away from its true mean is low. Therefore, when the estimate of the observed std. deviation of a large sample is low, then the mean value of the KPI calculated from the same sample is expected to be a good estimator (Ross 2013). To include the confidence level in his argument, Ross (2013) utilises the std. normal distribution as a multiplying factor to the observed std. deviation in the sample experiment.

Therefore to utilise Ross' (2013) approach, the first task is to define your accepted std. deviation, h . This can in the case of the Job Shop KPI be one minute similar to what was the case for the half-width in Rossetti's method. Thereafter, a pilot experiment of n_0 replications is conducted and the observed std. deviation, s , is calculated. The last required element is then the multiplying factor based on your required confidence level and accepted risk, α . The multiplying factor is calculated through the standard normal distribution as seen in eq. 5.4.

$$\Phi^{-1}\left(1 - \frac{\alpha}{2}\right) \quad (5.4)$$

5.2. REQUIRED NUMBER OF REPLICATIONS

With the multiplying factor calculated, the comparison is made to determine whether or not the n_0 replications are enough to ensure a std. deviation below the threshold value is illustrated in eq. 5.5.

$$\frac{\Phi^{-1}(1 - \frac{\alpha}{2})s}{\sqrt{n_0}} < h \quad (5.5)$$

If this inequality is satisfied, n_0 will ensure $(1-\alpha)\%$ confidence in the fact that the average KPI value calculated across the replications will not have a std. deviation of more than h . If the inequality is not satisfied, a new experiment with a higher n_0 must be conducted. As this iterative process can continue for an undefined number of iterations, the following practical approach is recommended in Excel.

1. Define the required threshold on the std. deviation, h , and the accepted risk level, α
2. Conduct a pilot experiment with 200 replications and export the KPI value from all the independent replications to Excel
3. Set $n_0 = 100$ and calculate the KPI's average across these replications and the experienced std. deviation
Use the **AVERAGE** and **STDEVA** functions in Excel
4. Utilise eq. 5.5 to make the comparison
5. If successful, the initial 100 replications are enough to ensure confidence in the calculated KPI average. If not, a higher n is required. Therefore select a new n that is either only 1 higher than n_0 (if the observed std. deviation is close to h) or a lot higher than n_0 (if the observed std. deviation is far from h). Now, go back to step 3 and continue the iterative process with the updated n
Having an initial set of 200 independent replications in one Excel file will allow you to simply update your calculations in Excel and not re-conduct a new experiment

5.2.3. Single formula derived from both Rossetti and Ross

The iterative process by Ross can, if not performed in the practical manner, require an unknown number of different experiments. To counter this, a single formula based on the standard normal distribution and an initial estimate of the std. deviation can be derived from both Rossetti (2016) and Ross (2013). The iterative process from Ross already utilises the standard normal distribution while the t-distribution used by Rossetti in eq. 5.2 also will converge to the standard normal distribution when n gets large (Rossetti 2016). It is therefore possible to utilise the single formula seen in eq. 5.6 to determine the required number of replications, n .

$$n \geq \left(\frac{\Phi^{-1}(1 - \frac{\alpha}{2})s}{h} \right)^2 \quad (5.6)$$

In the formula, s is the initial estimator of the std. deviation observed and it is, therefore, necessary to conduct a pilot experiment of $n_0 = 5, 10$ or 15 before the single formula can be utilised. Additionally, in regard to this single formula, it should be mentioned that it should only be used if n is large (e.g. $n \geq 50$), because it requires a large number of samples before the t-distribution converges to the standard normal distribution (Rossetti 2016).

With the single formula explained, the three approaches to calculate the required number of replications to ensure confidence in your decision-making have now been introduced. Which method to use will depend on one's preferences and practical setup.

To conclude the introduction of the approaches, it is important to mention that each method is calculating the required number of replications based on one single KPI. Therefore, if your simulation study is focusing on multiple KPIs, you must either conduct the calculations for all KPIs and select the largest calculated n as the final number of replications to ensure confidence across all KPIs. Alternatively, and the approach used in a more practical setting, you can select the most important KPI, based on your simulation study's overall aim, and use this when calculating n . Afterwards, it will then be possible to calculate a confidence interval for the other KPIs based on the approach described in sec. 5.2.3 that will still support your decision-making.

A last remark to the required number of replications is the connection between your desired half-width and the required number of replications. The smaller half-width you choose, the more replications will be needed to ensure confidence (Rossetti 2016). The task is therefore always to consider the actual system that you are simulating and the aim of the simulation study and through this find an acceptable half-width that will be best fitted for the situation at hand.

Ensuring confidence after the final production experiment With the number of required replications defined, the production run of one's model can be performed and used for decision-making. When calculating the required number of replications, one ensures confidence in the reported value of the most important KPI. However, because the system being modelled often can be described through multiple KPIs, it is relevant to be able to provide confidence for decision-making for these as well.

This confidence can be established by defining a range in which, it is possible to state that with a specific level of confidence, the true value of the KPI will lie. Such range for the KPI is possible to obtain through the use of the standard normal distribution (Ross 2013). It is because the central limit theorem states that the distribution of a random variable will converge to a normal distribution when the number of observations grows (Routledge n.d.). Combining this with the estimators for a KPI's mean and std. deviation obtained in the production experiment, it is according to Ross (2013) possible to calculate the interval in which the KPI's true value with a probability of P will lie within as seen in figure 5.4.

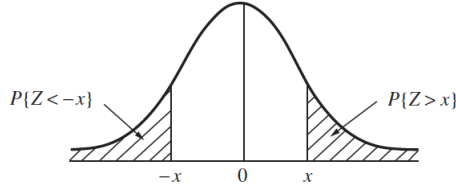


Figure 5.4.: Illustration of the confidence interval that can be calculated for a KPI through the std. normal distribution (Ross 2013, Figure 8.1)

In figure 5.4, Z is the random variable of a given replication of the model while x is a specific distance away from the observed mean value. To calculate this interval, the first task according to Ross (2013) is to define the confidence interval and related risk, α , similar to when the required number of replications were defined. With the confidence level defined, the interval can be calculated through eq. 5.7 that, as explained, is built on the standard normal distribution, the observed values in the production experiment for the KPI's mean, $\bar{x}$, and std. deviation, s , and the actual number of performed replications n .

$$\bar{x} \pm \frac{\Phi^{-1}(1 - \frac{\alpha}{2})s}{\sqrt{n}} \quad (5.7)$$

By applying eq. 5.7, it will therefore be possible to obtain the explained range for a KPI's value that one can be $(1 - \alpha)\%$ confident that the true performance of the KPI will lie within. This method from

Ross is valuable to establish confidence for decision making and a last note about its method is the resemblance of eq. 5.7 to eq. 5.2 in Rossetti's half-width approach. Actually, it is also the case that if the distribution of a KPI's value in the replications of one's experiment follows a normal distribution as seen in figure 5.3, the formula for the initial half-width h_0 can also be used to define the confidence interval for a KPI. This formula by Rossetti (2016) is, therefore, an alternative to the general case of using the standard normal distribution (Ross 2013).

5.2.4. Determining the number of replications - Practical Approach

The analytical method for figuring out the right size of the half-width is considered the correct approach. However, in practice, often and more simple way can be adopted for determining the right number of replications.

1. Run a fixed number of replications; start with 20.
2. Inspect the across replication statistics: look at the mean and half-width.
3. Try to increase the number of replications and look at the results. If the results change a lot, probably you need more replications; otherwise, if the results don't have a lot of variability and therefore the results are pretty much the same, you can choose depending on the desired half-width.

To visualize if a simulation has had enough replications, a steady-state plot can be made. A steady-state plot is a plot of the average over the number of replications. For example, the first value is the value itself, the second value is the average over the first two values, the third value is the average over the first three values (...) the last value is the average of all the replications values for that given scenario.

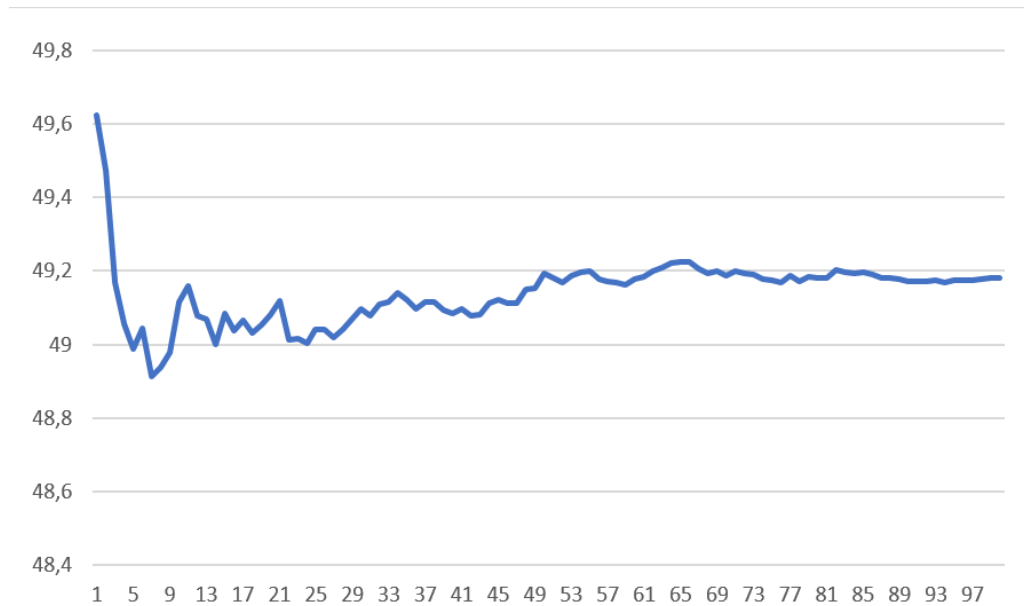


Figure 5.5.: Example of a steady-state plot of the Job Shop example

It can be seen after approximately 70 replications the values of the lead time become fairly constant, and that is when steady-state is achieved. To see how this is done look in the Job Shop example → ParameterVariation → Java Actions, and then look in the output Excel File.

5.3. Comparison of system configurations

As shown in the previous sections, conducting simulation experiments through replications can allow you to confidently make decisions based on true system performance. Another strength of simulations is the ability to quickly change your input parameters, e.g. the number of truck lifts in the Job Shop example, to allow you to efficiently test and compare different system configurations. When simulating various configurations, the calculated KPI values will allow you to compare the performance of the configurations for you to be able to design an optimal one for your business setting.

However, when comparing systems, it can also be beneficial to with confidence state that one configuration is statistically better than another in regard to a specific KPI. This could as an example be the case if you as an analyst wish to suggest to Hospital management that an MRI room should be replaced by an extra treatment room because this will increase patient flow without increasing patient wait time. When making such an argument, the ability to statistically state that patient wait time will not increase will highly strengthen your argumentation.

To build such an argument, you can utilise your simulation model, the single KPI in question and a common stream of random numbers. What is meant by a common stream of random numbers is the ability in AnyLogic to conduct two sets of n replications with the same degree of random noise affecting replication $i = 1...n$ in both set one and set two. For the previous sections, independent random number streams were used, because the aim was to ensure confidence in one final production run of your model and not in the comparison of different configurations of the model. Now, when the task is to statistically verify a difference between configurations, using a common random number stream across the same number of replications will allow you to compare the system performances and not the randomness of the number streams. A simple analogy to the comparison of two cars can facilitate the understanding of why common random number streams are needed. Because, when comparing the performance of two cars, you will always use the same track, the same conditions and the same driver so the only input parameter changed in the performance test is the car itself while all other parameters are kept the same.

With a common random number stream enabled, the task is to conduct an experiment of n replications for each of the two configurations and export the KPI values to Excel. For each replication i , the difference in KPI value between the two experiments is now calculated through eq. 5.8 whereafter the mean value of the differences is calculated through eq. 5.9 (Rossetti 2016).

$$D_j = X_{1j} - X_{2j} \quad \forall j = 1...n \quad (5.8)$$

$$\bar{D} = \frac{1}{n} \sum_{j=1}^n D_j \quad (5.9)$$

With the mean difference defined, an experienced half-width for the obtained mean can be calculated through the same approach as Rossetti's half-width method in sec. 5.2.1. First, the observed standard deviation is calculated through eq. 5.10 where after the $(1-\alpha)\%$ confidence interval for the difference can be defined by eq. 5.11 (Rossetti 2016).

$$S_D = \sqrt{\frac{1}{n-1} \sum_{j=1}^n (D_j - \bar{D})^2} \quad (5.10)$$

$$\bar{D} \pm t_{\alpha/2, n-1} \frac{S_D}{\sqrt{n}} \quad (5.11)$$

With the confidence interval defined, the conclusion on the comparison of the two systems can have three outcomes:

5.3. COMPARISON OF SYSTEM CONFIGURATIONS

1. If the entire interval is **above 0**, configuration one will be statistically better than configuration two in regard to the KPI in question
2. If the entire interval is **below 0**, configuration two will be statistically better than configuration one in regard to the KPI in question
3. If the interval **include 0**, no decision can be made on the comparison of the two configurations and a larger number of replications, n , is required

With this short process, it is therefore now possible for you to compare system configurations and with $(1-\alpha)\%$ confident state which system is better in order to strengthen your argumentation of system design.

To conclude the description of system comparison, it is important to mention the practical aspect of different system configurations. It is because if the difference between the two is small but one configuration has a smaller operation cost or a simpler implementation this configuration should of course be chosen. It is therefore always important to balance the practical aspect of your business setting with the statistical comparison of configurations to best suggest optimal system designs for a management group (Rossetti 2016). With the statistical comparison, you just have an additional tool available to strengthen your argumentation.

6. Experimental design

6.1. What is experimental design?

When the simulation model has been verified and validated, the simulation project is ready to advance to the third phase of the general simulation methodology, ie. to decide on the *Experimental design* (Rossetti 2016). Experimental designs usually take a starting point in the baseline model and perform a number of experiments that investigate the goals and objectives of the system (Rossetti 2016). In the experimental design terminology, the input parameters and structural assumptions composing a model are called *factors*, and output performance measures are called *responses* (Law 2015). When creating the Experimental design it is important to keep the goals of the study in mind as these will differ from one simulation project to the next. The goals of the study affect which system parameters should be considered fixed and which may be considered experimental factors. Later in this chapter, table 6.1 will provide an overview of experimental factors typically seen within operations management.

6.1.1. The operations management (OM) perspective

Simulation is a popular tool within OM when analysing a specific system as it offers the manager the ability to quantify and observe the system's behaviour. A simulation model allows the manager to explicitly visualise how operations are expected to perform under different inputs while examining the behaviour and responses (Montazer et al. 2003).

Having the OM perspective in mind when creating an experimental design is very important in order to obtain valuable information about a system's processes and its possibilities for change and optimisation. A part of the OM perspective is to set some *Key performance indicators* (KPIs) which are quantifiable measures of performance over time for a specific objective. The KPIs provide a target for the development of an experimental design.

6.1.2. The purpose of experimental design

The model's performance is highly dependent on factors and *system specification changes*. It is therefore essential to investigate how the performance measures changes dependent on a change in the input factors or the system and furthermore which factors that has the greatest impact. Therefore it is essential in phase 3 to design an experimental design that are consistent with the project's KPIs. The purpose of experimental design is therefore to estimate how changes in input factors affect the results or responses of the experiment. This is done by examining how a number of parameters or structural assumptions affect the performance measures or which set of model specifications leads to the optimal performance.

6.2. Creating an experimental design

When creating a scenario different factors in the system can be changed. It is often desirable to determine which factors have the greatest effect on a response with the least amount of simulating, therefore it is important to design the experiments carefully and thoroughly as a hit-or-miss sequence can be time-consuming and not as relevant.

One can distinguish between controllable and uncontrollable factors and qualitative and quantitative factors. The controllable and uncontrollable factors are dependent on whether they represent a controllable action for a manager in a real-world system. It is usually controllable factors in simulation experiments that are relevant as these reflect the decision about the implementation that has to be made in the real-world (Law 2015). However uncontrollable factors such as; asses how a 10 per cent increase in the arrival rate of customers would affect congestion can be interesting as well.

There are typically both qualitative and quantitative factors in a model that can be changed. A quantitative factor is a parameter such as the number of machines or resources in production while a qualitative factor is a model specification such as a route or a requirement in a surgery. The table below shows common qualitative and quantitative factors that are controllable in a model.

System	Possible factors	Quantitative or qualitative	Possible responses	Customers
Inventory system	Reorder point	Quantitative	Average cost per month, average number of items in inventory	Items
Inventory system	Order-up-to level	Quantitative	Average cost per month, average number of items in inventory	Items
Manufacturing system	Number of machines	Quantitative	Average time in system, throughput, average queue size	Parts
Manufacturing system	Number of forklifts	Quantitative	Average time in system, throughput, average queue size	Parts
Manufacturing system	Forklift dispatching rule	Qualitative	Average time in system, throughput, average queue size	Parts
Hospital system	Nodes of nurses, beds	Quantitative	Average time in system, throughput, average queue size	Patients (uncontrollable)
Hospital system	Number of doctors	Quantitative	Average time in system, throughput, average queue size	Patients (uncontrollable)
Hospital system	Hospital policy	Qualitative	Average end-to-end delay, throughput	Patients (uncontrollable)

Table 6.1.: Typical experimental factors used within operations management (Law 2015).

6.2.1. Scenarios

An experimental design evolves around a number of scenarios that should be carefully chosen to shed light on different aspects of the system examined. Each scenario is based on the baseline model where either an input factor (quantitative change) is changed or a system specification (qualitative change) is changed.

The scenarios can be presented in a so-called *scenario tree* as the one seen in Figure A.4. The scenario tree summarizes (in headlines) the main characteristics of the scenarios and illustrates how these relate to each other. In this way, the problem-owner is presented with an easy-to-read overview of the scenarios examined.

Conducting the simulation experiment (productions runs)

After having decided on the composition of the scenarios, it is now time to run the simulation model for each of the scenarios. The system is set up so that the number of replications decided upon, cf. chapter 5, is run with a random seed.

6.2.2. OFAT approach, factorial designs and optimisation

Sometimes a simulation model can have a lot of input factors and many KPIs to consider, leading to a complex experimental design if one would have to test each factor one at a time. This is both time-consuming and ineffective. In such cases other methods like factorial design (Law 2015) or simulation-based optimisation (Law 2015) are more effective as these gives can be oriented towards a specific objective and consider factor interactions and multiple combinations.

OFAT approach and factorial designs

There are different approaches that investigate the effects when changing an input factor.

If a model only has one factor the experimental design is simple, as the simulation would be to run with various values of the factor making it easy to determine the optimal value that gives the best response (Law 2015). But there is typically more than one factor in a model, therefore it can be rather complex to get an estimate of how each factor affects the response and how the factors interact with each other e.g. if the effect of one factor on the response depends on the values of another factor.

One simple way to measure the effect of a particular factor at the time is to fix the rest of the factors. This approach continues until all factors have been investigated one at a time. This strategy is called the one-factor-at-a-time (OFAT) approach. This strategy can be quite time-consuming and ineffective as many simulation runs are needed if many input factors are present. Furthermore, the approach does not take the combination of the factors into account and thereby neglects the effect of the interactions between the factors.

Another approach is the 2^K *factorial design* that considers more factors at a time ($K \geq 2$) and allows one to change several factors at a time. In a factorial design approach the effect of several factors and the interactions between them are investigated, enabling one to examine the extreme *design points*. In a factorial design, we determine two levels for each factor that creates two extreme design points. As only two levels are considered for each factor it is in this approach assumed that the response approximated linear (monotonic) over the range of the factor (Law 2015). It is important to determine levels that are realistic, but far enough apart to see a difference in the response.

There are many advantages of using factorial designs, first of all, factorial design detects interaction between factors leading to fewer misunderstandings of how the responses changes with the factors occur compared to OFAT. Furthermore, the factorial design allows the effects of a factor to be estimated at several levels of the other factors.

Simulation-based optimisation

Simulation-based optimisation are methods whose objective is to find a combination of input factors that optimises a key output performance measure. These input factors of interest could both be quantitative or qualitative variables depending on the overall goal of the study. This could for example be an output of economic importance such as profit or cost, or it could be an output of service level such as waiting time or error in production.

The optimisation method is built by creating an objective function based on selected output performance measures from the simulation that are dependent on the values of the input factors. These input factors also have a set of constraints/requirements that the simulation model needs to satisfy (a lower or upper bound), this could e.g. be that the average waiting time should be less than 5 minutes or that the utilisation of a resource should be a minimum 80%. Depending on the problem

the objective function can either be maximised or minimised over all combinations of the input factors that also satisfy the constraints.

Solving an optimisation problem using a simulation model can be quite difficult, as the problem quickly becomes large due to the input factors amongst others. Many simulation-based software include packages, which can optimise an objective function and search for the best combination of inputs. The software packages are often based on rather simple, yet efficient optimisation methods using various types of ascent/descent heuristics and in some cases metaheuristics.

In Anylogic, it is possible to create an optimisation experiment that enables one to search for a set of input factors corresponding to the best value of a given objective function. The objective function is created based on the responses and the overall goal of the simulation study. In the optimisation experiment, you can determine the minimum and maximum value that a parameter for a factor can take and if needed requirements for the responses can be added to ensure that a specific response always satisfies a given bound.

As described in chapter 5 it is important to plan your simulation study subject to the stochasticity that the model is exposed to. Thus, careful considerations on the number of replications needed should be made (cf. chapter 5).

6.3. How-to plan the experimental design

6.3.1. How-to guide

Following guide shows the steps necessary to design an experiment.

1. Prerequisites: Ensure base-line case has been verified and validated
2. Determine relevant input factors with respect to the problem statement
3. Determine possible responses and set up in the simulation model
4. Design experiments - factorial design
5. Output analysis of scenarios
6. Determine best scenario // Deep dive in scenario

Prerequisites

In phase 2 in Rossetti a baseline has been established representing the ground truths, that have been stress tested, validated and verified. This is a prerequisite before any experimental design can formally begin. When the model is validated and verified it is possible to begin on the experimental design to assess how sensitive the response is when changing input factors.

Determine input factors

The first step is to determine what input factors that are relevant to analyse in relation to the overall project statement and the established KPIs. If the objective is to analyse the operational performance in a surgery clinic with respect to reducing cycle time for patients, then relevant input factors could be the number of nurses and doctors and the number of operation rooms available, while if the objective is to optimise the flow in a manufacturing system, then relevant input factors are e.g. number and utilisation of machines.

Determine relevant responses

The next step is to determine possible responses of the input factors that are relevant to analyse and

add them to the simulation model. These are often based on the determined KPIs and could for example be the utilisation of nurses in a hospital system, average queue time or average time in the system for a product to be produced in a manufacturing system. See table 6.1. After having determined relevant responses these should be added to the simulation model, in order to perform a statistical analysis.

Create scenarios

The next step is to design the experiments through different scenarios. In this step, it is important to make use of empirical/experience-based knowledge of the specialists, managers and/or operators of the system being examined. An obvious starting point for the scenario design is to focus on the bottlenecks of the base case system.

If only one input factor is determined, then only one scenario is created. However, for more than one input factor it is more efficient to use factorial designs such that all combinations of factors are considered. Scenario 0 is the baseline scenario and the experimental designs are built from this scenario. To achieve an overview of the different scenarios it would be recommendable to create a scenario tree, which illustrates the created scenario in regards to the baseline.

For each scenario, it is necessary to run the simulation a suitable number of times to gain robust and reliable performance measurements. This can for instance be done using a parameter variation in *Anylogic* in which the number of replications and the seed is set to random (cf. eLearning module on replications). The parameter variation in *Anylogic* allows the modeler to let the number of resources (e.g the number of machines or nurses on a shift) take a number of different values within specified ranges..

Output analysis and best scenario

At last, the responses can be analysed to examine the performance measurements and determine the optimal scenario in relation to the problem statement. This is done as in the output analysis as described in chapter 5.

When determining the best scenario it's necessary to compare the performance of the different experimental designs in regards to the created KPIs. The comparison can be done in many ways depending on the complexity of the system and the project. Typically, for a complex system, some of the KPIs chosen may be more important than others. In such cases, an *AHP* or a *cost-benefit* analysis may be relevant. These two types of analysis enable comparisons of the scenarios with regard to the relevance of the KPIs.

7. Further reading

AnyLogic provides a online-library called "**AnyLogic** Help" explaining all functions inside the program. If you want to use blocks or functions not explained in this guide, you can use this.

Besides that **AnyLogic** provides a bunch of example models that touches several key areas of the program. Especially more complex systems. You can find these by selecting Help and then example models inside **AnyLogic** .

References

- AnyLogic (n.d.). *Job Shop model*. Last accessed 22nd March 2021. URL: <https://help.anylogic.com/index.jsp?topic=%5C%2Fcom.anylogic.help%5C%2Fhtml%5C%2Fprocessmodeling%5C%2Fjobshop%5C%2Ftutorial.html>.
- Law, Averill M. (2015). *Simulation Modeling and Analysis*. Fifth edition. MC Graw Hill.
- Montazer, M. Ali et al. (July 2003). "Simulation modeling in Operations Management". In: *The title of the book*. Ed. by The editor. Vol. 4. 5. <https://www.pomsmeetings.org/confproceedings/001/papers/psc-04.4.pdf>. The organization. The address of the publisher: Proceedings of the 14th Annual Conference of the Production and Operations Management Society, p. 213.
- Ross, Sheldon (2013). "Chapter 8 - Statistical Analysis of Simulated Data". In: *Simulation (Fifth Edition)*. Ed. by Sheldon Ross. Fifth Edition. Accessible through DTU Findit. Academic Press, pp. 135–152. ISBN: 978-0-12-415825-2.
- Rossetti, Manuel D. (2016). *Simulation Modeling and Arena*. Second edition. John Wiley Sons, Incorporated. ISBN: 9781118607916.
- Routledge, Richard (n.d.). *Central limit theorem*. Last accessed 19th April 2021. URL: <https://www.britannica.com/science/central-limit-theorem>.

A. Simulation project - example

In the following example, the steps associated to the creation of an experimental design of a small simulation study is described.

YummiBar's produces three different kinds of chocolate bars. The production process is outlined in Figure A.1. The company produce for 15 hours per. day, after which a cleaning team moves in and cleans the machines so they are ready for the next day.

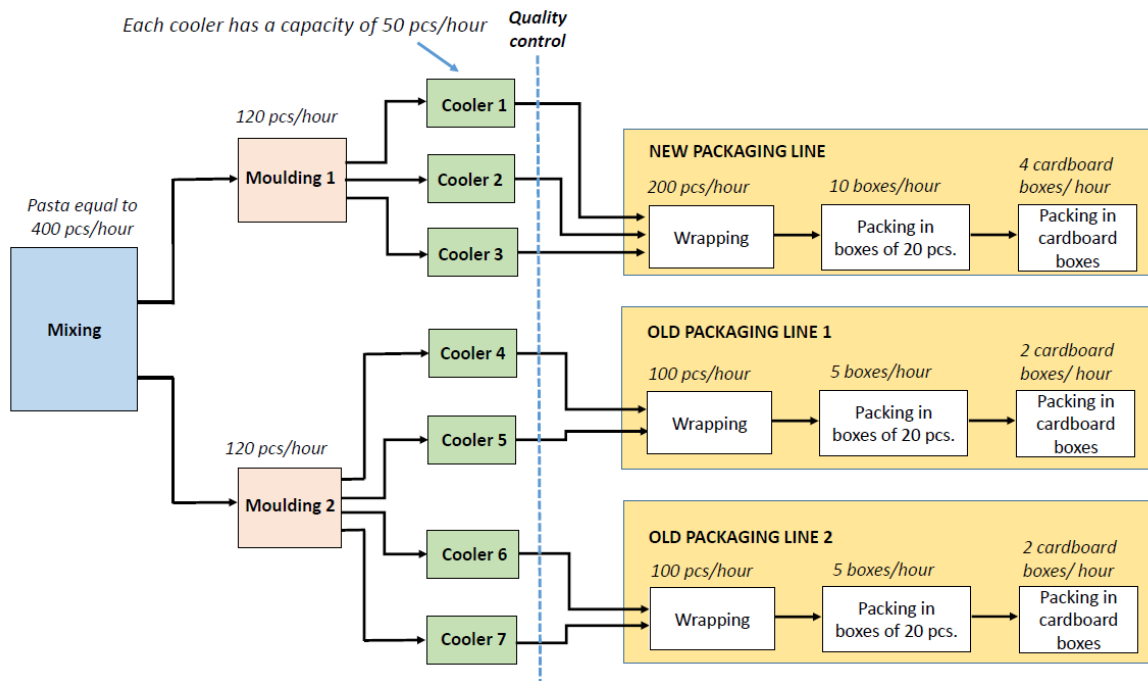


Figure A.1.: The baseline design of the YummiBar's production line.

As can be seen from the figure, the production process is divided into a total of six sub-processes;

1. mixing
2. moulding
- 3.

1) mixing, 2) moulding, 3) cooling, 4) wrapping, 5) packing in small boxes of 20 pcs. and 6) packing in cardboard boxes, where the small boxes are packed in large cardboard boxes with room for 20 small boxes in each. The first four sub-processes are automatic with only a little need for interaction with the operators in production. The last two sub-processes, on the other hand, are manual, where a number of packaging staff are responsible for the practical unpacking of the finished products. The last three sub-processes take place in parallel at two old packing lines and one new packing line. In addition, both "moulding" and "cooling" take place in parallel sub-processes with two moulding

machines and seven coolers. Furthermore some breakdowns unfortunately happens every 30 minutes to 1 hours on the wrapping machine which should be included in the model, one can consider the frequency of breakdowns as a uniform distribution between 30 minutes and 1 hours.

YummiBar's has a demand that varies throughout the year. The company has decided that they are aiming for a production of 3000 chocolate bars per. day.

A.1. Setting the objectives

The first phase of the general simulation methodology is to define the problem, set the objectives and plan the project. Subsequently, relevant KPI's in regards to the project statement should be established. In this very simple system, the following KPI's have been established to monitor the production of the YummiBar's such that the best output is provided.

- Reduced production time of the chocolate bars
- Maximum 80% Avg. utilisation of machines
- Reduced production cost

After having determined the KPI's the and stress-tested and validated the model, a base case has been established A.2.

With the base case as a starting point, we can begin the experimental design to investigate scenarios that are better. Following the guide in the prior section, the first step is to determine relevant input factors and possible responses.

In this system the input factors that are considered are:

- Number of machines
- Number of resources
- Number of chocolate bars

After having determined the relevant input the responses that we find relevant for the system in regards to the KPIs are defined.

- Utilisation of machines
- Time in system
- Average time in system

When responses and input factors have been established we can begin the experimental design properly. For each scenario an output of the responses is provided concatenating step 3 and 4.

A.2. Base case scenario

The base case scenario is the first model of the simulation and has been validated and verified, which means all 3000 chocolate bars are being produced each days and the model enables the requirements/restriction from the Yummibar's company's production. The performance of the system is shown in table A.1 and in figure A.2 and A.3 below.

A.2. BASE CASE SCENARIO

	Avg. time in system	Utilisation of moulding	Utilisation of cooling	Utilisation of wrapping	Utilisation of small box	Utilisation of cardboard box
Base case	9612,73sek	90,23%	60,14%	83,33%	49,99%	6,25%

Table A.1.: Scenario 1 KPI performance

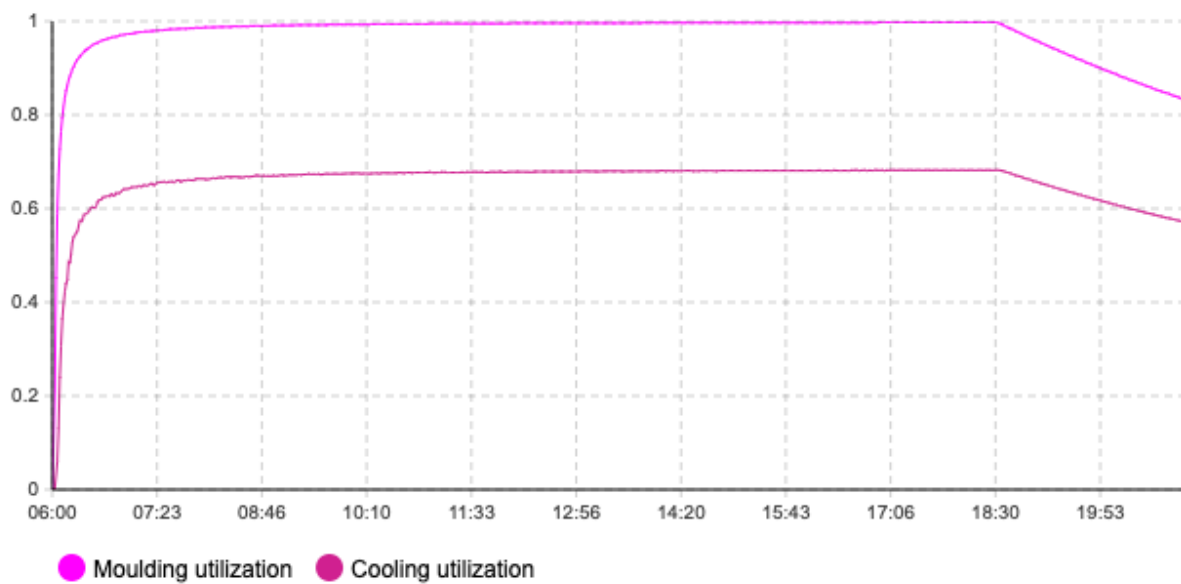


Figure A.2.: Utilization of moulding and cooling

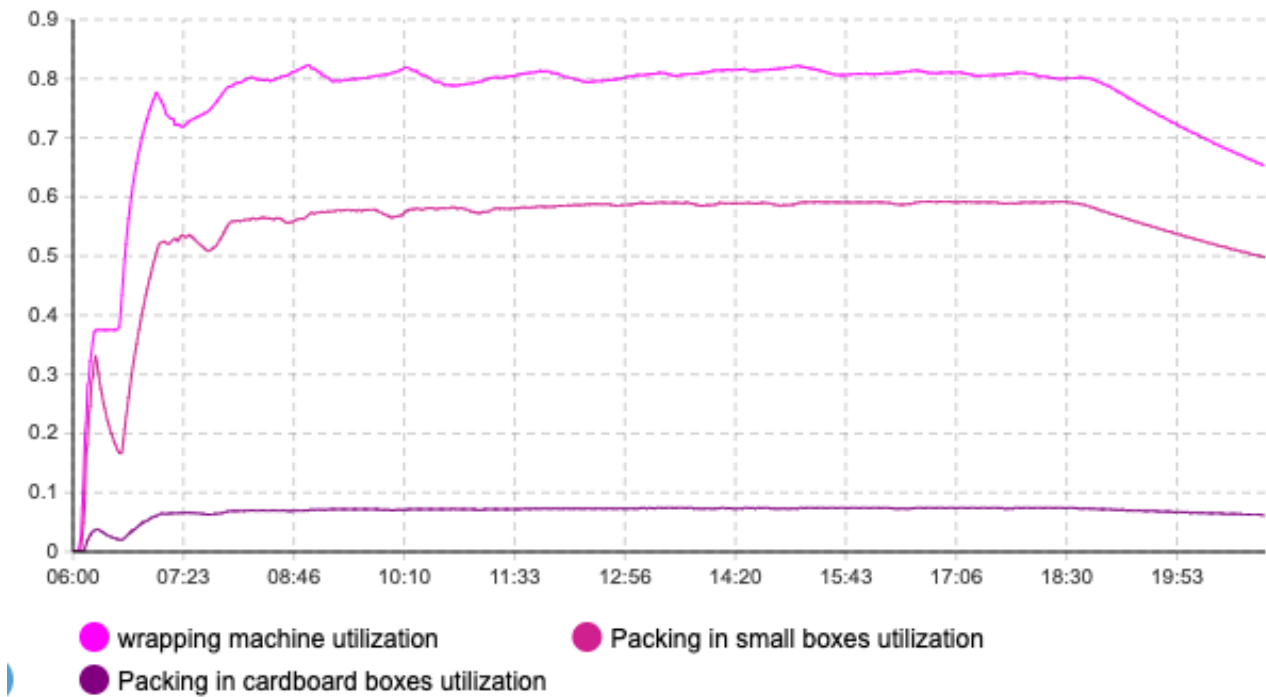


Figure A.3.: Utilization of wrapping, packing in small boxes, packing in cardboard boxes

It is obvious from Figure A.2 that the utilisation of the moulding machines is extremely high. This indicates that the moulding machines constitutes a significant bottleneck in the system as a large queue of chocolate bars occur before the machine. Furthermore, it can be seen that the utilisation of the wrapping machine is quite high due to the breakdowns. Lastly, it can be seen from Figure A.3 that the utilisation of the packing work in cardboard boxes is extremely low, which means that the resources are not used efficiently and it could therefore be relevant to investigate a new packing setup where the resource is used more efficiently leading to a higher utilisation.

From the base case a scenario tree has been created with different scenarios that hopefully would bring insightful findings and increase the performance of the defined KPIs.



Figure A.4.: Scenario tree

The different scenarios and results will now be further explained.

A.3. Scenario 1 - Predictive maintenance on wrapping machine

As the management of the YummiBar's has decided that the average utilisation of machines should not exceed 80%, it can be seen that the both the utilisation of the wrapping machine and moulding machine are too high. To decrease the utilisation of the wrapping machine, scenario 1 is therefore extended with a night technician in order to mitigate the probability of breakdowns on the wrapping machine during production time. It is assumed that if the wrapping machines are maintained after production hours, it will reduce the probability of a breakdown to between 2-4 hours which allows a better production flow with less disturbances. This does however give an additional cost of the extra technician when maintaining the machine. The results from the scenario can be seen below:

	Avg. time in system	Utilisation of moulding	Utilisation of cooling	Utilisation of wrapping	Utilisation of small box	Utilisation of cardboard box
Base case	9612,73sek	90,23%	60,14%	83,33%	49,99%	6,25%
Scenario 1	9500,93sek	90,23%	57,14%	53%	50%	6,25%

Table A.2.: Scenario 1 KPI performance

It can be seen that the night technician reduce the utilisation of the wrapping machine and thereby satisfy the determined KPI. The utilisation of the wrapping machines has decreased giving a more smooth flow and a reduced production time due to the fewer breakdowns. However, the model does still have an obvious bottleneck at the moulding machines which still needs to be considered.

A.4. Scenario 2 - New moulding machine

When examining the baseline model, the obvious bottleneck is the moulding machines which increases the production time for the chocolate bars mitigating the quality of the chocolate bars and thereby increasing the risks of scrapping the bars. The focus point of this scenario is to reduce the average production time and to mitigate the utilisation of the machines by using additional resources to invest in a new and powerful moulding machine that can mould the bars faster.

It is assumed that this machine can mould 400 bars pr. hour and is an additional resource in the new package line. This will of course bring a higher cost, however it is expected that the value from the new moulding machine eventually can balance the resources out. The model is adjusted accordingly by ensuring that a higher percentage of bars comes through the new moulding machine.

The responses can be seen in the table below.

	Avg. time in system	Utilisation of moulding	Utilisation of cooling	Utilisation of wrapping	Utilisation of small box	Utilisation of cardboard box
Base case	9612,73sek	90,23%	60,14%	83,33%	49,99%	6,25%
Scenario 2	3219,52 sek	59,35%	57,14%	86,82%	50%	5,83%

Table A.3.: Scenario 2 KPI performance

The new moulding machine increases the performance of the two KPIs. It can be seen from the table above that the average production time to is decreased with 2/3 and the utilisation level is evened out amongst the 3 machines.

Furthermore, the additional resource enables the simulation model to run for 12 hours instead of 15 hours as the new moulding machine make it possible to produced the 3000 chocolate bars within the 12 hours window. The decreased production hours would help equalising the extra expenses of the moulding machine as the company e.g. can save money on salary of the packing employees.

A.5. Scenario 3 - remove cardboard box machines

The third scenario is as 1 and 2 based on the base case and investigates the effects of removing one of the packing cardboard boxes resources such that the utilisation of the resource is increased and used more efficiently. This is contrary to the other scenarios, a quantitative change in the model, as it is

A.6. SCENARIO 4, COMBINATION OF SCENARIO 1 AND 2

a system specification that is changed. The scenario investigates how the model runs when removing two cardboard resources, such that only one resource is used to collect the small boxes in cardboard boxes. The responses can be seen below.

	Avg. time in system	Utilisation of moulding	Utilisation of cooling	Utilisation of wrapping	Utilisation of small box	Utilisation of cardboard box
Base case	9612,73sek	90,23%	60,14%	83,33%	49,99%	6,25%
Scenario 3	9488,86 sek	90,23%	57,14%	86,82%	50%	10,00%

Table A.4.: Scenario 3 KPI performance

From the table it can be seen that the system is not highly influenced by the subtraction of the two resources as the utilisation only increases with 3.75%. This qualitative change is therefore a valuable specification change that can decrease the overall cost as fewer resources are used. Looking further into this, it can also be evaluated in a scenario, if this resource needs to be available during the entire production or if a schedule would be more appropriate, such that the resource is utilised more than 10% leading to a further reduction of expenses.

A.6. Scenario 4, combination of scenario 1 and 2

Scenario 4 is based on a combination of scenario 1 and 2, in order to examine if the performance measurements improves when having both a new moulding machine and a predictive maintenance on wrapping machine.

	Avg. time in system	Utilisation of moulding	Utilisation of cooling	Utilisation of wrapping	Utilisation of small box	Utilisation of cardboard box
Base case	9612,73sek	90,23%	60,14%	83,33%	49,99%	6,25%
Scenario 4	3943,45sek	62,40%	57,14%	49,0%	57,14%	5,8%

Table A.5.: Scenario 1 KPI performance

From the performance metrics it can first of all be seen that the average time in system is cut in one third and the utilisation of the moulding machine is evened out as in scenario 2. Furthermore, it can be seen that the utilisation of wrapping has decreased due to the additional resource on the night shift.

A.7. Scenario 5, combination of scenario 2 and 3 (moulding and cardboard box)

Scenario 5 is based on a combination of the scenarios 2 and 3 to examine if the performance measurements improves when having a new moulding machine and removing one two cardboard machines. Removing resources from the cardboard packaging would help balance out the additional resources used to implement a new moulding machine.

The performance measurements can be seen in the table below.

	Avg. time in system	Utilisation of moulding	Utilisation of cooling	Utilisation of wrapping	Utilisation of small box	Utilisation of cardboard box
Base case	9612,73sek	90,23%	60,14%	83,33%	49,99%	6,25%
Scenario 5	3188,98sek	59,40%	57,14%	83,33%	46,94%	10,3%

Table A.6.: Scenario 1 KPI performance

As for scenario 2 and 4 the production time is reduced to a third resulting in a faster production time. Furthermore, it can be seen that the utilisation of the moulding machine has decreases while the utilisation of the cardboard boxes has increased. As stated in scenario 3, it could be relevant to look into a schedule of the resources used in cardboard boxes to increase efficiency and utilise the resource more.

A.8. Comparison of scenarios

After having established relevant scenarios and investigated the output responses the last step is to determine the best scenario in regards to the overall project objective.

This is done by comparing the scenarios created in the experimental design. In the table below the responses can be seen.

	Avg. time in system	Utilisation of moulding	Utilisation of cooling	Utilisation of wrapping	Utilisation of small box	Utilisation of cardboard box
Base case	9612,73sek	90,23%	60,14%	83,33%	49,99%	6,25%
Scenario 1	9500,93sek	90,23%	57,14%	53%	50%	6,25%
Scenario 2	3219,52 sek	59,35%	57,14%	86,82%	50%	5,83%
Scenario 3	9488,86 sek	90,23%	57,14%	86,82%	50%	80,00%
Scenario 4	3943,45sek	62,40%	57,14%	49,0%	57,14%	5,8%
Scenario 5	3188,98sek	59,40%	57,14%	51,0%	46,94%	8,3%

Table A.7.: Scenario 1 KPI performance

When evaluating the scenarios in regards to the determined KPIs it can be seen that scenario 2, 4 and 5 stand out due to the average production time in the system, which lead to a decrease in production time and therefore a reduced risk of misshaped chocolate bars. This benefits the overall production as fewer misshaped would decrease the cost in production.

The choice of the best scenario should be up to the management to decide, as they maybe think one of the KPIs are more important that the others.

A relevant response which is not included in this little example, would be to examine the queue/waiting time before each machine in order to quantify the bottlenecks in the system. In this example the obvious bottleneck was the moulding machine which could be seen in the Anylogic model, and a measurement on the size of this queue could maybe be relevant for the decision in regards to the experimental design and the managerial aspects of the final decision.

Another relevant aspect would be to quantify the cost of e.g. having a additional moulding machine, a night shift technician to fix the wrapping machine or the decreasing cost of removing to cardboard resources. The quantification of could make it easier to choose the best scenario. An AHP analysis could be used for this, this analysis can also take the potential ranking of KPIs into account.

B. Simplified activity diagram for the modified JobShop model used in chapter 5

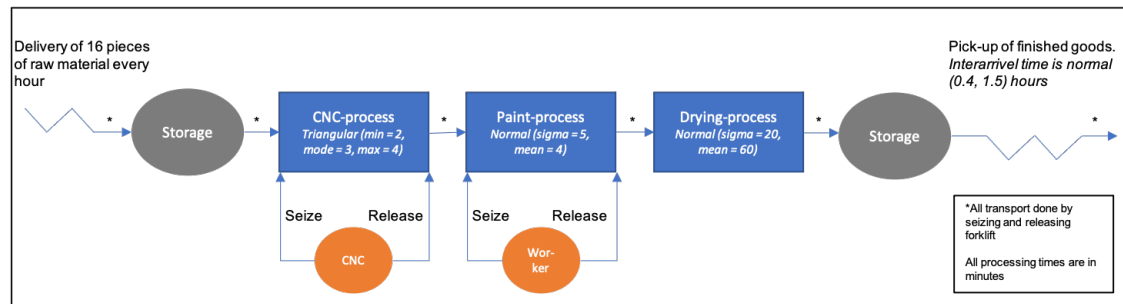


Figure B.1.: Simplified activity diagram for modified Job Shop model